



COMSM0128:

Immersive Games Design and Production

LIAR VR

Technical Report

Group B

Full Name	Student ID
Pranavv Jothinathan	2674319
Yifei Liu	2637548
Lintao Guo	2807006

Table Of Contents

DESCRIPTION OF EACH MEMBER’S CONTRIBUTIONS	4
Yifei Liu.....	4
Lintao Guo.....	5
Pranavv Jothinathan	6
1 INTRODUCTION.....	8
2 BACKGROUND.....	8
2.1 About Game	8
2.2 Game Introduction.....	8
2.3 VR Multiplayer platform.....	9
2.3.1 spirit sling	9
2.3.2 Unity VR multiplayer platform	10
2.3.3 Deciding to Develop from 0.....	11
2.4 PVP or AI	11
2.5 Hands or Controllers	12
2.6 Game Outro	12
2.7 Game Genre	12
2.8 Game Target Market.....	13
2.9 Discussion of Related Commercial Products and Research	14
2.9.1 Demeo: VR Tabletop Presence and Turn-Based Interaction	14
2.9.2 Among Us 3D: Embodied Social Deduction and Cross-Platform Deception.....	14
2.9.3 Cards & Tankards: Social VR Card Play and Tavern-Based Interaction.....	14
2.9.4 Research on Social VR Platforms and Presence	14
2.9.5 Research on Non-Verbal Communication and Feedback in VR	15
2.9.6 Summary: Design Implications for LiarVR	15
3 GAME SETUP	15
3.1 Game Rules	15
3.2 Win Conditions	15
4 GAME IMPLEMENTATION.....	16
4.1 Scene Environment.....	16
4.1.1 Asset Sources and Reused Materials.....	16
4.1.2 Self-Built Scene Layout and Design Details	16
4.1.3 Lighting and Technical Setup.....	17
4.2 Introduction UI and Player Guidance	17

4.3 Two players in the same scene	21
4.4 VR Camera	23
4.5 Character Model Import	25
4.6 Turn-based	28
4.6.1 Mode 1: Two players take turns playing cards until one player plays all 5 cards and wins.....	29
4.6.2 Mode 2: Player 1 Plays a Card, Player 2 Challenges	31
4.6.3 Hurry Up Sound Effect Button	32
4.7 Card Issue	33
4.8 Test of Multiplayer Games by One Person	34
4.9 End Game Sequence: Visual Effects	35
4.9.1 Result: Victory	35
4.9.2 Result: Defeat	35
4.9.3 Unity Particle System	36
4.9.4 Development of Fire Effect.....	36
4.9.5 Development of Smoke Effect	37
5 CRITICAL EVALUATION	38
5.1 Data Source and Method	38
5.2 Data Processing and Simple Model.....	39
5.3 Quantitative Results	39
5.4 Qualitative Feedback	41
5.5 Critical Evaluation and Reflection	42
5.6 Limitations and Future Work	44
REFERENCES.....	45
APPENDIX A	47
Consent Form	47
Questionnaire	47
Interview Questions.....	48

Description of each member's contributions

Yifei Liu

Game Development and Implementation

Area	Contribution
VR Game Study	• Evaluated <i>Spirit Sling</i> VR game for gameplay and interaction design
Platform Analysis	• Analyzed VR multiplayer platforms for suitable development approach
Game Design Decision	• Compared PvP vs AI and selected PvP gameplay model
Interaction Design	• Selected hand-based interaction over controllers
Framework Development	• Developed basic VR multiplayer framework
Multiplayer Setup	• Implemented two-player shared VR scene
Character Integration	• Integrated real-body tracking character model
Turn System	• Implemented turn-based gameplay system with event-based control
Issue Resolution	• Fixed card physics issues (gravity vs Photon network conflict)
Card Initialization Interaction and Reset	• Implemented card initialization, interaction and reset

Testing

Area	Contribution
VR Game Testing	• Tested <i>Spirit Sling</i> for gameplay flow and interaction study
Platform Testing	• Tested VR multiplayer platforms for suitability
Character Testing	• Tested VR body tracking and alignment
UI Testing	• Tested challenge button functionality
Gameplay Testing	• Tested turn-based system behavior
Animation Testing	• Tested animation execution and sync
Card System Testing	• Tested card interactions and identified issues
Multiplayer Testing	• Conducted overall multiplayer stability testing

Report Preparation

- Contributed to the preparation and documentation of the project report by explaining the technical workflow, interaction design, and implementation details of the developed modules.

Lintao Guo

Game Development and Implementation

Area	Contribution
Scene Environment Design	• Created VR tavern scene environment using reused assets • Adjusted spatial layout for comfortable VR interaction • Refined environment setup for improved usability and immersion
Introduction UI and Guidance System	• Implemented in-game introduction guidance system • Refined player instructions for first-time gameplay understanding
Card Dealing and Reset Flow	• Organized card dealing animation flow • Implemented round reset sequence • Improved consistency of round start and restart behaviour
Card-Box Interaction Optimization	• Improved card submission interaction with card box • Increased stability and reliability of card placement during gameplay
Round Logic Integration	• Connected physical card interactions with round logic system • Integrated player actions with turn-based gameplay updates
VR Interaction and Experience Design Support	• Contributed to shaping the overall user experience flow by supporting interaction and gameplay presentation decisions in the VR environment

Testing

Area	Contribution
VR Game Testing	• Tested <i>Spirit Sling</i> for gameplay flow and interaction study

User Study and Data Analysis

- Prepared and refined the prototype for user testing, ensuring that the main gameplay flow and interaction features were ready for evaluation.
- Prepared consent forms and translated questionnaires for Wenjuanxing data collection.
- Processed questionnaire results and interview responses.
- Built a simple evaluation model to summarize and interpret the user experience.

Report Preparation

- Contributed to the preparation and documentation of the project report by explaining the technical workflow, interaction design, and implementation details of the developed modules.

Pranavv Jothinathan

Game Development and Implementation

Area	Contribution
Platform Analysis	Analyzed VR multiplayer platforms for suitable development approach
Framework Development	Developed basic VR multiplayer framework
Introduction UI and Guidance System	Implemented in-game introduction for the players, along with instructions and effects for the player understanding.
Player Transportation System	- Implemented player transportation between gameplay and transition areas to improve scene flow - Created corridor-style transition experience to separate gameplay space from end-game feedback environment
End-Game Visual Effect Design	Designed the overall concept for win and lose visual feedback to enhance emotional impact at game conclusion
Unity Particle System Implementation	Implemented win and lose effects using Unity's Particle System with appropriate emission, color, and motion settings
VR Interaction and Experience Design Support	Contributed to shaping the overall user experience flow by supporting interaction and gameplay presentation decisions in the VR environment

Testing

Area	Contribution
Player Transportation Testing	Tested and refined player transportation system between gameplay and transition areas
Visual Effects Testing	Tested and refined the end-game visual effects for enhanced player feedback
Platform Testing	Tested VR multiplayer platforms for suitability
Character Testing	Tested VR body tracking and alignment
UI Testing	Tested challenge button functionality
Gameplay Testing	Tested turn-based system behavior
Animation Testing	Tested animation execution and sync
Card System Testing	Tested card interactions and identified issues
Multiplayer Testing	Conducted overall multiplayer stability testing

Data Analysis of User Study

- Processed questionnaire and interview responses to extract key user experience insights
- Summarized findings to evaluate usability, engagement, and overall interaction quality

Report Preparation

- Contributed to the preparation and documentation of the project report by explaining the technical workflow, interaction design, and implementation details of the developed modules.

1 | Introduction

The gaming world in the Virtual Reality (VR) are getting popular with a rapid growth in recent years mainly due to the advancements in the immersive technologies, head-mounted displays, and tracking systems. Virtual reality (VR) is increasingly used as a social platform for users to interact and build connections with one another in an immersive virtual environment.^[1] The reasons for the rapid growth could be highly realistic and interactive gaming environments which is not possible in the traditional gaming experiences. Among the Virtual Reality games, the Multiplayer Virtual Reality games have gained major attention among the users as the games offer collaboration, competition, and communication for the users inside the shared virtual environments.

“Sense of presence is an important factor influencing the quality of the virtual reality (VR) experience.”^[2]

However, the effective social interaction among the users inside the shared virtual environments remain a major challenge. The limited understanding of this affects the presence for users inside the games. Social Virtual Reality offers new forms of social interaction, identity expression, and embodied experiences, but it has also revealed significant issues related to social inequalities and unequal power dynamics within virtual worlds.^[3] Though our project does not exactly solve the social inequalities and unequal power dynamics, as of our understanding, our project uses a robotic avatar for the players which might not make the players feel social inequality or unequal power dynamics. According to Riches, S., et. al (2019), Sense of presence decreased when the players experienced diminished agency and experienced physical impediments.^[2]

In a way of addressing the limited social interaction in virtual environments, our project focuses on developing a multiplayer virtual reality card game, LiarVR. The game is designed in a way that it encourages active communication, strategic thinking and interaction, and socially immersive experience. The game encourages the players to get involved into the flow of the game, as the players must observe, think, plan strategically throughout the game.

The project also explores to improve the player involvement, communication inside the virtual environments, and thus the overall immersive experience for the players. The game attempts to explore the potential of virtual reality by creating a game with socially interactive gaming experiences.

2 | Background

2.1 About Game

Released in 2024, Liar's Bar is a multiplayer deception game set in a shady bar where bluffing and outsmarting your opponents is key to survival.^[4] The card game in which you attempt to get rid of all the cards in your hand, bluffing along the way. Players have to decide when to call someone's bluff and if you are caught cheating, your virtual character will then 'shoot themselves' - and the aim is to be the last one standing. It features cartoony anthropomorphic animal characters like a pig in a vest shirt, a female fox, a Rottweiler-style dog with a gold chain and a bull in a polo shirt - think Zootopia but darker.^[5]

The game comes with 2 different modes, Liar's Deck and Liar's Dice. The Liar's deck is a card-based strategy game mixed in with Russian roulette. The goal of the game is the deceive and outsmart other players and be the last one living. Liar's Dice is a dice-based strategy game. The goal of the game is to gamble your chances and be the last one living.^[6] While the original game has two modes: Liar's deck and Liar's dice, our game consists only of Liar's deck.

2.2 Game Introduction

LiarVR is a 2-player VR card bluffing game that takes place in a tavern-like virtual world with real physical cards. Players are both sitting at a virtual table with their cards laid out on the table. LiarVR also provides an experience of embodied interaction; thus when players look around, reach for cards to play, put cards

into play, and respond to their opponent through challenges, they engage with LiarVR differently than if they were just viewing a flat interface of cards on a screen.

A player starts the game on a Meta Quest headset, an online multiplayer room is created via Photon PUN 2. The experience is for two players; once the two players are connected they then participate in a shared card game session. The project was created in Unity with support from VR Meta XR and used Photon's network functionality to synchronize the main flow of the game so both players see the same state of the game and can interact with each other in real-time.

Players receive a small hand of cards at the beginning of each round. In VR, they can select and move them in real life. When they take their turn, they will commit a card by putting it in the card drop box. This is their claim to have played that card and give the other player the ability to challenge it. The challenges are based on the possible bluffing involved when completing the game rules. The combination of VR hand interaction, the multiplayer connection, turn-taking, and bluffing rules may confuse new players on what to do right away. For that reason, the final version of the game will include an in-game introduction and guidance system to illustrate the basic flow of play through step-by-step explanations (e.g.: check to see if both players are ready, determine whose turn is next, put a card in the box, determine when a challenge can occur).

2.3 VR Multiplayer platform

Currently, there are several VR multiplayer platforms to choose from. We conducted many researches and experiments to determine which VR multiplayer platform to use as the foundation for our game.

2.3.1 spirit sling

Our initial idea was to improve based on existing multiplayer VR games. Since we had no systematic training previously in VR multiplayer game development, we wanted to use the networking function already implemented in other projects. Then we add our own game logic. This served two main purposes. Firstly, we wanted to verify the feasibility of secondary development on an existing VR multiplayer game. Secondly, we also hoped to learn from existing projects how to implement network visuals, player synchronization, and multiplayer collaboration mechanisms in VR multiplayer games.

First, we searched for examples online and found Spirit Sling through the Oculus sample. ^[7] Spirit Sling is a mixed reality social game that allows players to place a virtual chessboard on a real-world table and invite friends to join virtually. During gameplay, two players interact face-to-face using new Meta avatars, taking turns controlling the board and engaging in fun, sling-like battles on the table.

There are 2 reasons for choosing it. Firstly, it's an example provided by the Oculus sample, which should be relatively stable. Secondly, it's open source. We can download it directly from GitHub for further processing.

After downloading the project, we first attempted to understand the game structure, especially its networking logic. It is important for us to understand the state synchronization mechanism between different players in turn-based gameplay.

However, we found that the project's complexity far beyond our expectations. First, the Unity editor itself contains many call relationships between components and objects. And with different scripts, prefabs, and scene objects interconnected, it makes the overall structure difficult to understand. Due to the large number of components and the complex call chains, it was difficult to clearly define flows of each part in a short time. Understanding the network-related scripts was also challenging. The core network script alone had about 1000 lines of code, and it called many other components.

However, the project's functional direction was highly similar to our needs. It was a VR turn-based two-player game. The project would initialize a table with interactive objects on it, which was related to our players interacting with cards around a table. The project also displays the actions of another player and has implemented a turn-based system. Therefore, it has some reference value in terms of its functional goals.

Therefore, we decided to first experience and test the actual functionality of the project. If its functionality was stable, we would then try to understand the code structure and assess whether it was suitable for further development.

However, during testing, we discovered some practical operational issues with the project. For example, tables in the game would sometimes suddenly fly into the air after initialization and fail to return to their normal positions. What is more, the interactive responses of some objects were not sensitive enough, affecting the overall experience. Furthermore, the project contained many interactive objects and interaction methods, while our project only required relatively focused interactions around the cards. We were concerned that deleting or modifying a large amount of existing interactive logic would create additional workload. And it would potentially cause new compatibility issues.

In addition, the project leaned more towards an MR scene than the VR scene we hoped to achieve. Considering the project's complexity, operational stability, differences in interaction requirements, and development costs, we finally decided to abandon this.

2.3.2 Unity VR multiplayer platform

Realizing that directly modifying existing projects was impractical, we turned to finding VR multiplayer game development solutions. By searching for Unity's initial project templates and related resources, we discovered the Unity VR Multiplayer Platform, which provides fundamental support for VR multiplayer game development. ^[8]

Then we found a relevant tutorial on YouTube. This tutorial demonstrated how to create a simple VR multiplayer game based on the Unity VR Multiplayer Platform. Compared to directly modifying complex existing projects, this approach is more suitable for us to start building from the basic functions. And it also helps us gradually understand the player connection, scene synchronization, and multiplayer interaction logic in VR multiplayer games. ^[9]

However, when we tried using the VR multiplayer platform following the tutorial, we quickly met an unsolvable problem. After opening the multiplayer platform project in Unity, the console displayed errors, primarily related to Photon Fusion Voice.

Initially, we thought it might simply be due to incorrect settings, so we tried checking and adjusting the project configuration. We compared the steps in the YouTube tutorial and consulted the Unity official documentation on the VR Multiplayer Platform. But we still couldn't find an effective solution. Next, we went into the Photon platform and tried creating and entering the corresponding Voice App ID, hoping to resolve the issue by supplementing the Photon Voice configuration. However, even after adding the Voice App ID, the same or similar errors continued to happen in the Unity editor.

If we temporarily ignore the errors related to Photon Fusion Voice, the platform could play. Therefore, we first tested its basic functionality. The test results show that the platform can support two players entering the same room, and both can see each other's actions. This indicates that it works correctly in terms of basic multiplayer connectivity and object synchronization.

However, the platform example doesn't demonstrate the turn-based game logic we need. The scenario it provides is more like a simple multiplayer shared room where two players can synchronize their actions. Our card game requires a clear turn-based flow. This turn-based mechanism isn't entirely consistent with the synchronous interaction in the example.

Furthermore, this platform suffers from a problem like the previous Spirit Sling project. The network logic remains quite complex. While some scripts are only a few hundred lines long, they call many Unity components. Unity's own multiplayer platform contains many interactive components, each composed of multiple objects and scripts, resulting in a complex overall structure. For us, fully understanding these call relationships in a short time and adding our own turn-based card game logic remains a significant challenge.

Therefore, after trying existing projects and the Unity VR multiplayer platform, we began to rethink. Under the current capabilities and time constraints, what implementation method should we adopt to. We need both retain the VR card interaction and avoid being slowed down by an overly complex network system.

2.3.3 Deciding to Develop from 0

After several attempts with existing projects and platform solutions, we consulted with senior student Kun Yang for relevant experience. Kun Yang was an MSc student two years ago, and his project at that time was also a two-player VR game. It is primarily based on gesture-based interactive effects. Therefore, he had some practical experience in VR two-player game development.

In our discussions, he advised us to build the project from 0, rather than continuing to try to modify existing projects or complex platforms. He believed that our project was not large in scale, just a relatively small two-player VR card game. It did not need to use an overly complex off-the-shelf framework. Deconstructing and modifying existing projects might waste a lot of time understanding other people's code and component structures.

He also recommended two YouTube tutorials for reference. One on implementing online two-player FPS shooting games, and the other on VR bodytracking settings. He believed that these two tutorials would allow us to learn the basic logic of multiplayer online games and how to build a VR body, thus gradually establishing the basic framework for a two-player VR game. These tutorials were also resources they referenced during their project development. ^{[10][11]}

Therefore, we finally decided to develop the project from 0. That is, instead of modifying a complex existing VR multiplayer project, we started with a basic Unity 3D project, gradually building multiplayer connectivity, VR cameras, card interaction, and turn-based logic. While this required us to complete more basic functions ourselves in the early stages, the overall structure was clearer and easier for us to understand and control the project logic, especially the implementation of the turn-based process.

2.4 PVP or AI

Early in the project, we discussed a key question: should the opponents in the game be played by real players or by a intelligent AI? This question was first raised by the tutor during the initial pitch, so we specifically considered it and designed two possible solutions.

If we choose to use an intelligent AI as the opponent, the overall nature of the game will change. No matter how intelligent the AI appears, it will still make judgments based on preset rules or algorithms. We once thought a solution. We could design the game as multiple levels, each with a different AI behavior pattern.

Players would need to observe clues in the room or engage in combat with the AI to gradually explore and summarize its behavioural patterns. This would add elements of puzzle-solving and strategic exploration to the game.

However, this design direction was not related with our initial project goals. Our core idea isn't about players deciphering AI behaviour patterns. Instead, we aim to enhance fun of game through VR's unique embody interaction. Players could suspect each other through their body language. This adds even more fun in VR. This experience is difficult to provide in traditional 2D or 3D games, and it's what we believe is VR's greatest value.

Therefore, we prefer that the opponents in the game be real player. Interactions between real players have a greater degree of uncertainty and psychological game-playing. Players not only need to judge based on the cards, but also observe their body language, hand gestures, and on-the-spot reactions to deduce whether their opponents are bluffing or lying. In other words, the core enjoyment of our game lies in the interaction through virtual body, creating a psychological battle of wits between real players. It is a constant struggle to determine whether their opponent is a cheater for both players.

2.5 Hands or Controllers

While most VR games use controllers as the primary interaction method, this is usually because the game requires movement or teleportation. Due to the limited real-world space in VR, players would find it difficult to move long distances. It is difficult for large virtual maps without controllers to provide teleportation or movement. Therefore, controllers are a necessary input method in many VR games with large scenes or exploration content.

However, our game's scene is small and doesn't require additional movement. The core space of the game is just a table, and players primarily perform upper body movements and hand interactions within a limited range around the table. Therefore, we don't need to implement teleportation functionality, so we can play without controllers for movement control.

2.6 Game Outro

The game ends with an outro sequence, which would be different for either of the players. It was designed this way to enhance the player engagement and feedback. The player who won the game would be presented with a celebratory audio with a fire particle effect created using the Unity's particle system, and with a text acknowledging the victory. The player who lost the game would be presented with a motivational audio along with a smoke particle effect created using the Unity's particle system, and with a motivational text, to keep the players engaged towards the next game.

2.7 Game Genre

LiarVR is a multiplayer VR social deduction card game that uses cards as the basis for the game and includes three main elements of play: 1) using cards as a means of strategy; 2) using cards for social deduction; and 3) interactive use of immersive virtual reality technology.

LiarVR is a game where you play cards based on their shapes rather than suit, so the players have the Queen and King but can only play them as they would in real life. The rules of LiarVR are simple enough that anyone can learn them instantly, yet there is enough strategic depth involved to require careful consideration of timing and bluffing. When one player plays a card, it is up to the other player to determine

if that card was played truthfully or as a bluff. Because of this, LiarVR has a psychological aspect that makes it unique from physical card games.

Social deduction games pose a challenge that is non-mathematical or mechanical in nature, they are primarily social or interpretative. Players must deduce their opponent's intent with little information about their opponent. Since the players' cards are hidden from view (except for their own) there is always an element of uncertainty associated with each turn. The challenge mechanic is where players turn their suspicions of their opponent into action. Players can either guess incorrectly, resulting in a loss, or guess correctly, resulting in a win. Due to the high-stakes nature of this decision-making process, players remain focused on the logic of the gameplay and on how their opponent plays the game.

LiarVR is a VR card game. Different from a normal card game played online, every player's position is different rather than being a single name on a screen. Players sit in a shared tavern environment, use hand motions to move cards, and look through their avatars to see the game. LiarVR provides a much more realistic experience than online card games. Furthermore, LiarVR uses virtual reality technology to make the table, cards, play box and opponent real entities in the same physical space, instead of interface elements on a monitor.

2.8 Game Target Market

LiarVR is designed for casual and mid-tier VR gamers who like to play short replayable multi-player games. The game is ideal for Meta Quest gamers since it was developed for stand-alone VR headsets and has natural hand interaction. Unlike larger scale VR games, where you may need long periods of time to play, or complex controls, or lots of physical movement; LiarVR will be played while seated, at a table; therefore, it will be easier for people who are new to VR, or prefer to play social games, as opposed to action-packed games.

Another group of players who will find interest in the game are fans of social deduction games, bluffing games, and party games. Existing commercial games like Among Us, Liar's Bar, Poker, and many traditional card-based bluffing games have demonstrated that there are many people who prefer to play games that consist of suspicion, lying/deception, and reading other players. While keeping this design philosophy as a basis, LiarVR has adapted the game so that it is played in a much more personal 2-player manner in virtual reality. Since only 2 players can play this game, it creates a much more intimate experience with each player's decisions immediately impacting the other player. Additionally, the other player's reaction to those decisions create another essential aspect of the experience. Overall, this provides for a smaller, but highly engaging gameplay loop that lends itself to numerous iterations over multiple rounds.

Students, young adults and groups of friends will likely also want a VR social experience that's easy to use and lightweight. The game mechanics are relatively straightforward; therefore no extensive gaming experience is necessary in order to understand your objective. An in-game tutorial panel provides players with an overview of the gameplay, such as how to wait for another player, how to take a turn, how to watch the other player, and how to challenge a card that has already been played, therefore, making the game more user friendly for first time users and reducing the learning curve associated with most VR games.

2.9 Discussion of Related Commercial Products and Research

2.9.1 Demeo: VR Tabletop Presence and Turn-Based Interaction

Demeo is a useful comparison because it proves that tabletop mechanics can work effectively in VR. The game recreates the feeling of gathering around a physical game board with friends, using miniatures, dice, turn-taking and spatial manipulation as the basis of interaction.^[12] For LiarVR, Demeo is relevant not because the rules are similar, but because it shows the commercial strength of “being around the table” as a VR experience. LiarVR follows the same design principle at a smaller scale: the central table, cards and play box are not just interface elements but spatial objects that structure the player’s attention. This supports the project’s choice to focus on seated interaction, short rounds and readable physical actions instead of large-scale locomotion.

2.9.2 Among Us 3D: Embodied Social Deduction and Cross-Platform Deception

Among Us 3D is another important commercial reference because it shows how social deduction can become stronger when players are represented in a first-person or embodied 3D space. Innersloth describes the game as a 3D version of teamwork and betrayal with support for both VR and PC players.^[13] This suggests that deception games can benefit from spatial presence, voice communication and avatar behavior, because players are not only processing rules but also interpreting other people’s actions. LiarVR applies this idea to a smaller and more direct two-player card format. Instead of group deduction, the focus is one-to-one suspicion: after a card is played, the opponent must decide whether the action looks honest or deceptive.

2.9.3 Cards & Tankards: Social VR Card Play and Tavern-Based Interaction

Cards & Tankards is a stronger direct comparison for LiarVR than a general social VR platform because it combines VR, card play, multiplayer social interaction and a tavern-like fantasy atmosphere. The game is described as a social collectible card game available for both PC and VR, where players collect cards, battle other players or AI, and experience the feeling of playing in a local card store or tavern-style social space.^[14] This is relevant to LiarVR because both projects use a shared table, visible player presence and card-based decision-making as the center of the experience. However, Cards & Tankards focuses on collectible card-game strategy and long-term deck building, while LiarVR focuses on a shorter two-player bluffing loop. This difference helps define LiarVR’s unique value: it does not try to compete through card quantity or complex deck systems, but through psychological tension, hidden information, turn-taking and reading the opponent’s embodied behavior in VR.

2.9.4 Research on Social VR Platforms and Presence

Recent research supports LiarVR’s focus on social presence and shared interaction. Wei et al.’s 2025 systematic review of 94 HCI publications found that VR is increasingly used as a social platform and can support relaxation, engagement, intimacy and accessibility, while also raising challenges around safety, social norms and user experience.^[15] This is directly relevant to LiarVR because the project is not only a card-game prototype but also a small social VR environment. The game tries to create presence through a shared table, synchronized player bodies, visible card actions and turn-based decision-making. These design choices match the research view that social VR depends on both technical synchronization and meaningful interpersonal interaction.

2.9.5 Research on Non-Verbal Communication and Feedback in VR

Research on non-verbal social interaction in VR also supports the project's emphasis on clear gestures and feedback. Biggs et al. discuss the importance of non-visual and non-verbal interaction channels in virtual reality, especially for making social interaction more accessible and meaningful.^[16] In LiarVR, non-verbal information appears through hand movement, card placement, turn timing, challenge actions and end-game visual effects. These cues are essential because bluffing does not depend only on spoken explanation; it also depends on how confidently or hesitantly a player acts. However, this research also highlights a limitation of the current prototype: if card grabbing, play-box detection or network synchronization is unstable, the social meaning of the action becomes unclear. Therefore, future development should improve feedback when a card is accepted, when a challenge becomes available and when the round result is decided.

2.9.6 Summary: Design Implications for LiarVR

Together, related products and research suggest that LiarVR has a clear design space. Demeo proves the commercial value of VR tabletop presence, Among Us 3D shows how embodied social deduction can strengthen deception, and Cards & Tankards demonstrates that card-game interaction can work effectively in a social VR tavern-style setting. The research literature further supports the importance of social presence, avatar design, contextual guidance and non-verbal feedback. Therefore, LiarVR can be positioned as a compact two-player VR bluffing card game that combines commercial ideas from deception games and VR tabletop games with research-informed design choices around embodiment, accessibility and social interaction.

3 | Game Setup

3.1 Game Rules

At the start of the game, each player would have 3 Queen cards and 2 King cards, 5 cards in total. Player A starting the game can either play a Queen card which is the right card to be played meaning honesty or could play a King card which is not the right card meaning lie. Next, Player B could either play a card having two options: King or Queen or if player B suspects the first player lied by playing a King card, then player B can challenge the player A of lying by pressing the challenge button. The game continues with the same options for the next playing player, here player A.

3.2 Win Conditions

A player can win the game by:

- ✓ Emptying the hand meaning playing all the five cards and not getting caught by the opponent when playing the King cards.
- ✓ Successful challenge meaning by catching the opponent when the opponent plays a King card. Un-successful challenge would result in losing the game for the person who challenged.

4 | Game Implementation

4.1 Scene Environment

This section covers the environment part of the LiarVR Project (tavern scene, environmental props, lighting, layout, and render-pipeline setup). (Gameplay systems - cards, buttons, player avatars, and multiplayer logic - are not included here, as these are addressed elsewhere in the project). The environment is built in Unity 6 using the Built-in Render Pipeline. The final scene is a low-poly fantasy tavern, with an arrangement suitable for compact VR play. Although simply opening an example scene is not the focus of actual project development, earlier prepared self-created project materials and tavern-style assets were used to operate as a base, layout, and lighting redesign, to create a proper environment to support the needs of a two-player VR card game.

4.1.1 Asset Sources and Reused Materials

The main reference for the tavern resources is the Bar 02 model pack documentation provided with the project materials. The documentation lists compatible low-poly tavern models such as tables, chairs, benches, bar stools, cups, wine bottles, barrels, carpets, wall frames, cork boards, candles, chandeliers and railings. These assets were used as the environmental material source for the tavern style. In the project, these resources were treated as reusable building blocks rather than a finished scene. The actual environment composition was created by arranging, selecting and modifying these materials inside Unity. Only the assets that matched the LiarVR scene were kept, while unnecessary sample-scene elements were not used. This allowed the final environment to remain consistent with the previous project style while still being adapted to the new VR gameplay layout. Static decorative objects, such as tables, stools, cups, bottles and carpets, were mainly used to build the visual identity of the tavern. More structural assets, such as railings, wall objects and lighting props, were placed more carefully because they define the boundaries and atmosphere of the floating tavern space. The central table area was kept visually clear so that players could focus on the card interaction and the play box during the game.

4.1.2 Self-Built Scene Layout and Design Details

The scene layout was designed around the player experience in VR. The table is placed as the main interaction focus, with chairs and surrounding props arranged to make the space feel like a real tavern without blocking the player's view. The surrounding barrels, bottles, stools and wall decorations are used as supporting details, giving the scene a stronger sense of place while avoiding unnecessary clutter near the gameplay area. The environment also uses railings and edge props to communicate the limits of the floating scene. This is important because VR players need to understand the physical boundary of the playable area. The scene therefore balances two needs: it should look like a complete fantasy tavern, but it should also remain readable and comfortable for interaction. The color and lighting mood were adjusted from the original sample-scene feeling into a more suitable low-poly tavern atmosphere. Earlier versions had a stronger purple tone, while the revised environment uses a warmer and more controlled lighting direction so that the table area and props are easier to read. This helps prevent the environment from competing with the cards and other gameplay objects.

4.1.3 Lighting and Technical Setup

The Bar 02 documentation was used to configure the lighting resources correctly. For objects that only need to appear as static props, the original models can be dragged directly from the Models folder. For lights such as candles or chandeliers, the prefab versions are more appropriate because they contain the required glow and lights-on/lights-off components. The project uses the Built-in Render Pipeline setup. Following the documentation, the setup option under Tools/Fries/Bar 02 can create the FriesPropsManager object, connect the camera used for glow effects, set the post-process layer and initialize the lighting system. In the final scene, these effects were used selectively. The goal was not to enable every available effect, but to keep only the lighting that improves atmosphere and readability. The documentation also describes the Array Tool and Fence Tool. These tools were useful as editor-side layout helpers for repeated environmental elements, especially railing or prop placement. They are not runtime gameplay systems. Once the scene objects are generated and arranged, the final scene depends on the placed objects themselves, not on the tools continuing to run during gameplay.

Reference—Fries and Seagull. Bar 02 Model Pack Documentation. Provided project resource documentation, covering asset list, model usage, lighting setup, effects scripts, Array Tool, Fence Tool and render-pipeline switching method.

The main issue in the environment section is how to describe the asset work accurately. A more accurate description is that the scene was self-built using reused tavern resources, with custom layout, prop selection, lighting adjustment and VR-focused scene composition.

4.2 Introduction UI and Player Guidance

The Introduction UI was created to help new players learn LiarVR's gameplay and flow during their first round of play. Each of the four game mechanics involves connecting with other players using multiplayer multiplayer, taking turns, placing cards, bluffing and challenging, so players could easily be confused about how to play (especially if they are not provided with a basic understanding of your game outside of the game itself). As a result, all information about how to play has been provided through the world space as a front-facing virtual panel, which is visible to players in VR.^[17]

The two components used for the system's implementation are the IntroductionUIController and the IntroductionTutorialConfig. The function of the IntroductionUIController is to create and manage the world-space UI panel, to place it in relation to the player's head anchor, to update the text being displayed based on the state of the game, and to allow the player to hide the panel from their view. The Introduction tutorial configuration is used to store the tutorial text in different configurable stages/settings, so that the actual text content can be modified by making changes to the configuration rather than having to rewrite all of the core code. This allows for the tutorial system to be easily maintained and changed.



Figure 1: the Waiting for Both Players and Challenge Time stage

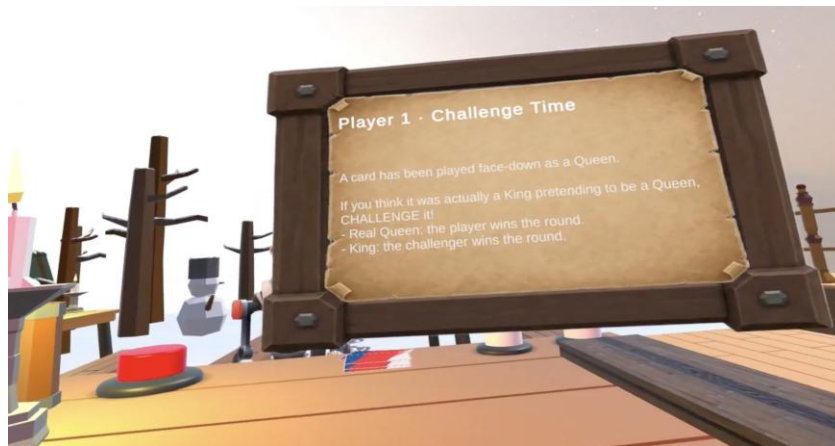


Figure 2: the Waiting for Both Players and Challenge Time stage

There are different stages in the guidance being divided as follows: Welcome, Waiting for Both Players, Both Players Ready, Your Turn, Opponent's Turn, Challenge Time, and Game Over. At the beginning of LiarVR (the name of the two-player VR card game that the guidance is for) there will be a panel that tells you about the game and also asks you to confirm that you can see the table, your cards, and the play box. During the Waiting For Another Player stage of the game, the panel will inform you that both players need to join the same Photon room. The Confirmation step of the Both Players Ready stage will confirm that both players can see the same table, cards, and play box.

When the turn changes, so does the UI. The player's turn will display "Your Turn" along with directions for selecting a card from their hand to be dropped into the rectangular play area. The opponent will view "Opponent's Turn" with instructions to watch the actual play of that card and to determine if they will challenge that play after the card has been placed. Role-related information will help players in a multiplayer VR game by providing different information to them during different periods of time.

The challenge rule is introduced in the context where players can apply it. When a player has played a face-down card as a Queen, the player will be informed they may challenge that card if they believe it is actually a King. Also at that time, the players will be informed of the outcome of the challenge: if the card is a real Queen, the challenger will win the round; if the card is a King, then the challenged player will win. By

showing the rule at this time, it becomes easier for players to learn and play the game, without having to memorize all of the details before starting the game.

Players can remove the tutorial by pinching/touching with their thumb and finger or pressing select during testing. By doing this there is no longer a blocker taking away the players ability to see the full game while playing. As an important aspect of VR interface design, the balance between providing guidance and allowing for immersion is achieved. A player has access to enough information to feel comfortable with what they need to do, but they can also choose to hide the panel if they want to be more focused on the game. Clear contextually appropriate guidance will help to learn and not confuse a player, but overuse UI may hinder a player's ability to be present through the use of VR.^{[18][19]}

Introduction UI is a state-driven tutorial system instead of a single static instructions page. The IntroductionTutorialStage enum represents the main tutorial flow in code with the following 7 stages: (1) Welcome, (2) WaitingForPlayers, (3) BothPlayersReady, (4) LocalTurn, (5) RemoteTurn, (6) ChallengeAvailable, and (7) Finished. These 7 tutorial stages correspond to the 7 main gameplay flow stages so that the UI displays different guidance depending on what is happening in-game.

```

2 个引用
public enum IntroductionTutorialStage
{
    1 个引用
    Welcome,
    0 个引用
    WaitingForPlayers,
    0 个引用
    LocalTurn,
    0 个引用
    RemoteTurn,
    0 个引用
    ChallengeAvailable,
    0 个引用
    ChallengeResolvedWin,
    0 个引用
    ChallengeResolvedLose,
    0 个引用
    Finished,
    0 个引用
    BothPlayersReady
}

```

Figure 3: Tutorial Stage Enum

IntroductionTutorialConfig has all the contents of the tutorial stored in separate text files. Each tutorial item has specific information about its target stage, target audience, title, body (text), and visibility option. When developing a piece of software, the way in which a user interacts with the UI is stored as a data file within Unity; therefore allowing developers to change how this type of guidance appears to users without changing any logic in an existing C# script.

The configuration asset is in Unity's Resources folder & loaded at runtime using Resources.Load(). If the tutorialConfig isn't assigned via Inspector, IntroductionUIController calls Resources.Load for "IntroductionTutorialConfig" to get the ScriptableObject automatically. This allows for the tutorial system working, regardless of when the controller is created dynamically at runtime, since the tutorial data can still be resolved from the Resources directory.

```

/// <summary>Loads Assets/Resources/IntroductionTutorialConfig.asset when the Inspector field is unset.</summary>
1个引用
private void TryLoadTutorialConfigFromResources()
{
    if (tutorialConfig != null)
        return;

    tutorialConfig = Resources.Load<IntroductionTutorialConfig>(TutorialConfigResourcesName);
}

```

Figure 4: Load TutorialConfig And Tutorial Step Config

Implementing a responsive UI to turn state for multiplayer games, allowing the active local player to see LocalTurn and the remote player to see RemoteTurn. Providing role-based Instructions, both players receive different directions during each phase of play; one player (the active turn) receives play a card instruction and the other (the observer) receives instruction to either challenge.

```

3个引用
public void SetTurnActor(int actorNumber)
{
    currentTurnActor = actorNumber;

    if (PhotonNetwork.LocalPlayer != null && actorNumber == PhotonNetwork.LocalPlayer.ActorNumber)
        SetStage(IntroductionTutorialStage.LocalTurn, true);
    else
        SetStage(IntroductionTutorialStage.RemoteTurn, true);
}

```

Figure 5: Turn Actor Switch

A panel will be created as an object in the real world, positioned in front of the user's headset in a world space to represent both the forward direction of the player from the head anchor and the set distance and height of the panel towards the player at a specific rotation; ensuring that the text remains legible but continues to provide an experience of the UI being part of the VR environment rather than a 2D graphical overlay.

```

/// <summary>
/// Positions the world-space Canvas directly in front of the HMD at <see cref="panelDistance"/>,
/// facing the player. Each player sees their own panel because the controller is local-only.
/// </summary>
4个引用
private void PositionPanelInFrontOfHead()
{
    if (headAnchor == null || worldCanvas == null)
        return;

    Vector3 forward = headAnchor.forward;
    forward.y = 0f;
    if (forward.sqrMagnitude < 0.001f)
        forward = Vector3.forward;
    forward.Normalize();

    Vector3 targetPos = headAnchor.position
        + forward * panelDistance
        + Vector3.up * panelVerticalOffset;

    worldCanvas.transform.position = targetPos;
    worldCanvas.transform.rotation = Quaternion.LookRotation(forward, Vector3.up);

    panelPositionInitialized = true;
}

```

Figure 6: World Space Panel Position

The tutorial's visibility is controlled strictly. Key parts of the tutorial forceShow to allow essential instruction, like prompt for first turn or explanation of challenge, to show while panel is hidden. The player can still close or toggle the panel, so that UI does not take a permanent space on the table. This is important in VR because large UI objects may block table objects, cards, or the play box. Therefore, implementation strives to provide guidance only when appropriate, while keeping focus on player-to-player interaction.

```

/// <summary>
/// Detects a sustained pinch (thumb + index tip) on left hand.
/// Holding for <see cref="pinchHoldDuration"/> seconds toggles panel visibility.
/// </summary>
1个引用
private void HandlePinchGesture()
{
    if (!enablePinchGesture)
    {
        pinchHoldTimer = 0f;
        return;
    }

    bool pinching = IsHandPinching(UnityEngine.XR.XRNode.LeftHand);

    if (pinching)
    {
        pinchHoldTimer += Time.deltaTime;
        if (pinchHoldTimer >= pinchHoldDuration)
        {
            ToggleVisible();
            pinchHoldTimer = 0f;
        }
    }
    else
    {
        pinchHoldTimer = 0f;
    }
}

```

Figure 7: Pinch To Close

This design separates the tutorial and UI content from the gameplay state detection. For example, IntroductionTutorialConfig manages the tutorial text and tutorial stage data; IntroductionUIController manages the creation and display of the tutorial UI panels; and Photon and TurnManager manage the information about the runtime state of the system. Because these components are independent of each other, the system is easier to debug and easier to be extended. For example, if you later needed to add a new stage to the tutorial (e.g., a rematch prompt or a more detailed explanation of the challenge results), you could simply add the new stage to the tutorial as another tutorial stage without having to rewrite the entire UI system.

The technical implementation serves the design goal of improving the understanding of LiarVR in the first play. The system incorporates configurable tutorial data, displayed by local player, makes Photon room detection occur, provides turn-based prompts, helps place panels in the world space and allows for manual dismissal. These options create an adaptive introduction rather than a static rules page and are better suited for multiplayer VR games.

4.3 Two players in the same scene

First, we searched and compared solutions for implementing multiplayer online gaming in Unity. We found Photon to be a commonly used network synchronization solution. What is more, multiplayer game functionality could be built from scratch in a Unity project by downloading the relevant libraries. Photon Fusion is a relatively new version, primarily used for synchronizing player states, object positions, and game events over the network.

After trying Photon Fusion, we found that it provides a built-in network synchronization mechanism. Developers can use the components and scripts provided by Fusion to synchronize player objects, scene objects, and related events. This approach is somewhat like the event-triggered workflow in Unity. In other words, we don't need to write network communication code from scratch. Instead, we can leverage Fusion's existing network components to achieve basic multiplayer connections and synchronization.

While searching for Fusion tutorials, we also found a type of tutorial similar to the "Gorilla Tag" style for synchronizing VR cameras or player objects. These tutorials demonstrate how to synchronize the head positions, hand positions, or body movements of different players over the network. Since our project also

requires two players to appear in the same scene and be able to see each other's actions, these tutorials are of some reference value for us in building a two-player VR scene.^[20]

Following the tutorial, we discovered that the camera setup used was unsuitable for our project. The tutorial used Gorilla Rig, a VR camera structure like that in the Gorilla Tag game. However, in practice, we found it difficult to find camera resources that were completely same to Gorilla camera in the tutorial.

Although the tutorial author provided relevant project files on GitHub, the Unity version used in the project was incompatible with ours. This situation prevented us from directly opening or reusing the camera settings. We then tried searching online for similar Gorilla cameras or player rigs, but most were paid content.

In addition, we tried finding free model resources and importing them into our own VR camera. However, the quality of free Gorilla models available online was unsatisfactory. Some models only showed the upper body, failing to meet the requirements for a complete player representation. And others were heavily pixelated, not matching the visual effects we wanted to achieve. Therefore, while this approach seemed feasible in the tutorial, it encountered significant difficulties in terms of resource acquisition and version compatibility.^[21]



Figure 8: New Gorilla Tag Rig

So we turn to another tutorial provided by Kun Yang, using Photon PUN2 instead of Photon Fusion.^[11]

Although this tutorial itself isn't a VR project, it clearly demonstrates how to use basic Photon scripts to allow two players to enter the same Unity scene and achieve basic multiplayer connectivity.

This part is extremely helpful. Although the later sections on shooting, weapons weren't directly related to our project, the fundamental logic of Photon room creation, player joining and player object generation is exactly what we need to learn. Through this tutorial, we can first understand how to make two players appear in the same Unity scene, and then build based on this to add VR camera.

In terms of implementation, we primarily utilize Photon's built-in network mechanism and write basic scripts to enable two players to enter the same scene. We referenced Photon's official introductory resource.^[22]

And after downloading and importing the relevant Photon libraries, we began building a basic online test scene.

First, we wrote scripts to connect to the Photon server and generate player objects. The core logic utilizes Photon's initialization and instantiation mechanisms to generate player objects. After the project runs, Photon retrieves the specified prefab from the Resources folder and generates it in the current scene.

```
PhotonNetwork.Instantiate(playerPrefabName, spawn.position, spawn.rotation);
```

Through testing, we are currently able to place two cube prefabs in the Resources folder and have the cubes corresponding to the two players appear simultaneously in the same scene.

During implementation this, we also met some issues. First, the Photon prefab used for network instantiation must be placed in the Resources folder, not just in the scene object. Otherwise, Photon cannot correctly find and generate the prefab using the correct path. Secondly, when testing two-player connections, we initially encountered a situation where only one local prefab was displayed in the scene, and the other player object was not visible.

To resolve this issue, we systematically investigated Photon's connection process using debugging information and breakpoints. Since the project was still a standard Unity 3D test scene at the time, we could directly add Debug.Log output to the script and use breakpoints to check the progress. We focused on several key callback functions in the Photon connection process, including "OnConnectedToMaster", "OnJoinedLobby", and "OnJoinedRoom".

```
PhotonNetwork.ConnectUsingSettings();
```

```
PhotonNetwork.JoinLobby();
```

```
PhotonNetwork.JoinOrCreateRoom("test", options, TypedLobby.Default);
```

By continuously setting breakpoints, we discovered the problem lay in the "OnConnectedToMaster" phase failing to complete correctly.

Finally, we discovered the cause was quite basic. the Photon App ID hadn't been correctly entered into the project. Since Photon requires an App ID to connect to the corresponding cloud application, room flows cannot be entered correctly without it. After creating the application and configuring the App ID in the Photon backend, we ran the project again. Then the connection process continued normally, with player objects being generated synchronously.

We could see two cube prefabs in the scene, but couldn't confirm whether their movements were truly synchronized. Therefore, referring to a YouTube tutorial, we added a simple keyboard movement script to the cubes. It allowed players to control their cube's movement through the keyboard.

The final test results show that both players' cubes can appear in the same Unity scene, and both players can see the movement and changes of the other's cube. This proves that we have initially achieved the function of two players entering the same scene in a Unity 3D game and completing basic position synchronization.

4.4 VR Camera

After completing the basic Unity 3D two-player synchronized test, our next step is to gradually transform the original 3D project into a VR project. The key to this process lies in the change of the camera. In the original 3D test scene, we used a Main Camera. And the player object was a cube prefab, which could be directly placed in the Resources folder and generated and synchronized by Photon after entering the room.

However, in a VR project, the camera is the core of the entire player's perspective and interaction. VR cameras differ from regular prefabs. They typically rely on a headset, tracking system, and VR components in the scene. Therefore, they cannot be simply placed in the Resources folder and generated directly by

Photon like a regular player prefab. In other words, VR cameras should be placed directly in the scene and run by the local device.

To address this issue, we separated the local VR camera and the network-synchronized prefab. Specifically, we placed a local VR camera in the scene to handle the current device's headset view and local input. At the same time, the player prefab was initialized by Photon after the player enters the room. This way, each client runs its own local VR camera and simultaneously generates the corresponding player prefab through Photon. In other words, our project structure contains only one VR camera and one player prefab. But when two clients run separately, they each initialize their local camera and synchronize two player objects through Photon.

The course primarily introduced how to build a VR camera using Meta XR Rig. However, after practical testing, we found that directly using this solution required adding and configuring numerous additional components to complete camera initialization, subsequent interactive functions, and character model import. This increased development complexity.

Therefore, we referenced tutorials on YouTube regarding the Meta Movement SDK and VR Body Tracking setup, and attempted to use the RigOVR Camera from the Meta XR All-in-One SDK. Compared to the XR Interaction Rig in the course. The RigOVR Camera comes with more complete Building Block components, making it easier for us to configure the VR camera, implement hand interactions, and rig character models. Therefore, we ultimately chose to build the VR camera system in our project based on the RigOVR Camera.^[23]



Figure 9: InteractionRigOVR-Basic

To achieve network synchronization, the objects in scene requires two core components, Photon View and Photon Transform View. Photon View assigns an ID to each player and acts as the main controller to manage the Photon Transform View. Photon Transform View controls child object nodes. These spatial nodes that need to be synchronized need to be manually added to the Photon View's watchlist to achieve realtime data synchronization through the network.

Additionally, a Wi-Fi connection is essential during the experiment because Photon uses a remote networking mechanism. If the device is not connected to Wi-Fi, local data will fail to upload to the network, resulting in errors and preventing the second player from appearing in the scene.

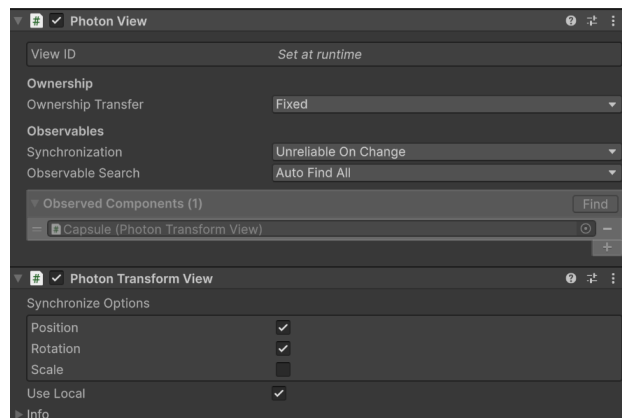


Figure 10: Photon View Component

Currently, we have successfully implemented two VR players in a scene using RigOVR and Photon view. Since RigOVR natively supports building blocks, integration was very convenient. Final test results show that not only can two blocks now be seen in the scene, but players can also move them using the controller and simultaneously see each other moving in realtime.

4.5 Character Model Import



Figure 11: the character model

Currently, the project only uses the VR camera, which can only display controllers or virtual hand models. However, this is far from expectation. To enhance immersion, we want to fit a complete character model onto the player's real body and make it appear in the game scene. The purpose of this is to allow players to show their status to other players through body language. So players could guess whether the other player has played the correct card or not.

To achieve this goal, we referred to the YouTube tutorial "Setup VR Bodytracking with Meta Movement SDK". In the specific implementation, we first imported the Meta Movement SDK into the project, and then configured the VR camera with body tracking functionality. For the character model, we used the classic Robot Kyle model, which is available for free in the Unity Asset Store.^[10]

Movement SDK	>	Body Tracking	>	Add Character Retargeter
Movement	>	Face Tracking	>	Create Retargeting Configuration
Properties...		Networking	>	Open Retargeting Configuration Editor

Figure 12: Body Tracking in Unity for VR^[24]

However, a serious problem arose during testing. In a single-player environment, body tracking worked perfectly. But when two players entered the scene simultaneously, both players' models incorrectly tracked the local player's movements. Specifically, if Player 1 entered the scene first, followed by Player 2. From Player 1's perspective, both robot models were synchronously tracking their own body movements. However, from Player 2's perspective, both models were completely still.

To find the cause of this problem, we conducted debugging for a long time. Initially, we suspected an incorrect import or configuration of the Unity Movement SDK. Perhaps the video tutorials we referenced

used an older version of Unity, whose SDK integration method differed from the current version. However, after cross-testing, we confirmed that the current SDK configuration was completely correct. And body tracking worked correctly in other single-player scenes. Then, we turned our attention to the scripts and the Photon networking mechanism, thoroughly examining the initialization scripts and the core principles of the Photon View and Photon Transform View. The results showed that the script logic was very simple, and there were no logical flaws in the network synchronization layer. In addition, we also investigated the compatibility of the camera hardware prefab and considered whether to replace the Oculus camera with the Meta XR Rig. However, since the camera performed perfectly in single-person testing, we ruled out the possibility that the camera itself was the problem.

To clarify our troubleshooting approach, we examined the system's core architecture. First, the scene contains a local VR camera and a player prefab. The system instantiates the prefab in the scene using Photon, and the VR camera drives and modifies the prefab in real-time using motion capture data, thus achieving body tracking. When a second player joins, they initialize a second prefab in the scene using their own camera. At this point, there are two cameras and two prefabs in the scene. And theoretically, their displacement and pose synchronization should be entirely handled by the network Photon View and Photon Transform View.

Since both models are tracking the same player's body, the problem clearly lies in Photon's synchronization mechanism being overridden by some local logic. It caused network synchronization to fail. We first performed `Debug.Log` in the script to confirm that all Photon callback functions were executing normally without throwing any network errors. Then we further confirmed that the Photon component itself was working correctly. Because position synchronization was completely normal when using blocks to represent the player, and the conflict only occurred after introducing body tracking. We thought the problem must point to the newly added body tracking component.

Based on this idea, we disassembled and disabled the body tracking components on the camera one by one for testing. We finally discovered that disabling the Character Retargeter component caused the opposing player's model to stop erroneously tracking local movements, and revert to a static state. So the problem was pinpointed to the Character Retargeter component. This component ignored the network hierarchy at runtime, incorrectly retargeting all model meshes in the scene onto the local player's skeletal data.

To address this component conflict, we considered leveraging Photon's access control mechanism. Based on our previous research of the Photon API, the built-in `photonView.IsMine` property of the component can accurately determine whether the current instance belongs to the local player. Therefore, we added a conditional check to the control script. During model initialization, the Character Retargeter component is activated to track body movements only if `IsMine` is true. Otherwise, if it's a remote player instance, the component is disabled. Through this dynamic access control, we successfully resolved the component conflict issue, achieving the desired effect of two players simultaneously present and each accurately tracking and synchronizing their own movements.

```
characterRetargeter.enabled = pv.IsMine;
```

After successfully resolving the motion tracking synchronization issue, a new problem arose during system operation. The size of the initially generated humanoid robot model was extremely unstable, sometimes displaying normal body proportions, and sometimes enlarged or shrunk.

To address this issue, we delved into the underlying mechanisms of the Character Retargeter component. Because this component uses a Humanoid model architecture, it includes an automatic scaling function. It

allowed the model to dynamically scale between 0.8 and 1.2 times based on detected data during initialization. Our initial instinct was to disable scaling, which ensured the model size remained perfectly consistent. However, this introduced another problem. Disabling scaling resulted in identical model sizes for players of different heights in the virtual scene, preventing the model from accurately fitting the player's real body and contradicting the project's goal of enhancing immersion.

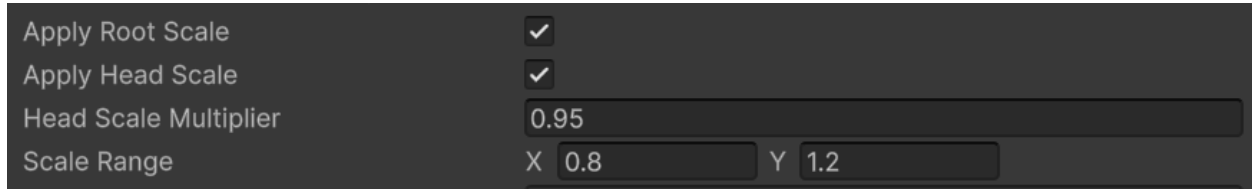


Figure 13: Character Scale

Therefore, to ensure the model's fit to the real body, we decided to keep the auto-scaling function enabled and began testing of external environmental variables that could cause scaling instability.

First, we conducted combined tests on the player's initial posture, simulating "both players initializing standing," "both players initializing seated," and "one standing and one sitting." The test results showed that simply controlling the player's physical posture did not result in a stable model size. We then speculated that this might be related to the wearing status of the headset during initialization. In subsequent comparative tests, we found that because the system automatically loads the model and aligns the data based on the ground height or reference plane detected by the headset during initialization, the player must wear the headset properly during script initialization.

However, after ensuring proper headset wear, while the model size stabilized during online testing with the same headset model, inconsistencies persisted when using different headset models. Further investigation revealed the root cause lay in the headset's internal Boundary settings. Experiments showed that both devices needed to have their boundaries set to "Stationary Boundary". If one device mistakenly used "Dynamic Boundary," the initialized character model would be significantly smaller.

In addition, the players' physical postures should be the same during initialization. To avoid size differences between the two players' models due to different initial height references, both players need to maintain either a sitting posture or a standing when initializing.

4.6 Turn-based

To clarify the core logic of turn-based games, we first need to define the control sequence of the entire game process using a flowchart.

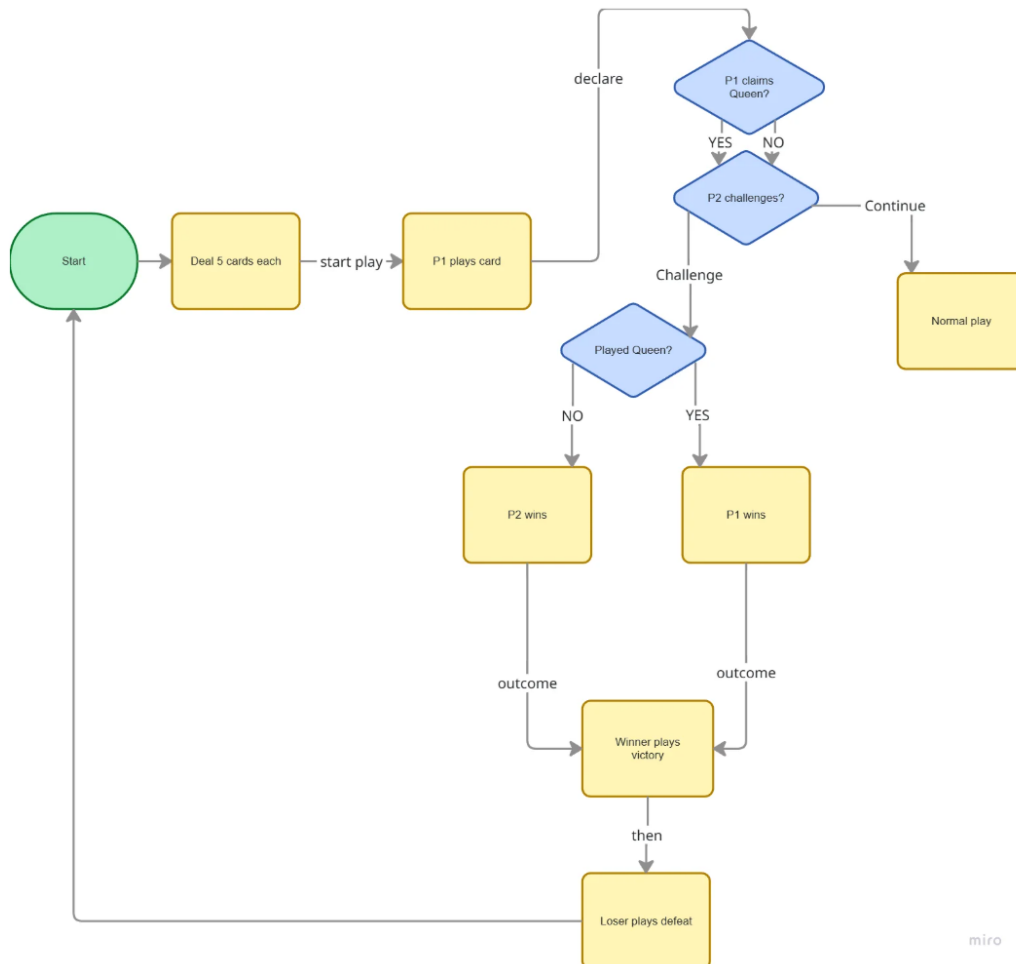


Figure 14: Game Logic, made by miro.ai

Currently, the project has successfully initialized the player model and cards. Next, the game officially enters the turn-alternating phase, and the primary task is to handle Player 1's card-playing logic.

To determine whether a player has executed a card-playing action, we have set up a specific physical area in the scene, temporarily using a box as the visual and physical carrier of the card-playing area. The core function of this area is as a detection trigger zone. Logically, we need to configure a collider on this box to prove that the player has correctly placed the card into the area.

Once the detection area detects the card's entry event, the system will trigger the following related logic. First, read and record the card's data information. Second, determine that Player 1's turn has been completed. Finally, synchronize this state through the network and officially transfer the card-playing authority to Player 2, then driving the rotation of the turn-based process.

4.6.1 Mode 1: Two players take turns playing cards until one player plays all 5 cards and wins.



Figure 15: Mode 1

In a normal round, the two players took turns playing cards according to the game rules. After a player placed a card into the play area, the play area uses a trigger to detect objects entering the area.

And it attempted to obtain the CardIdentity from that object to confirm that it is a Queen or King.

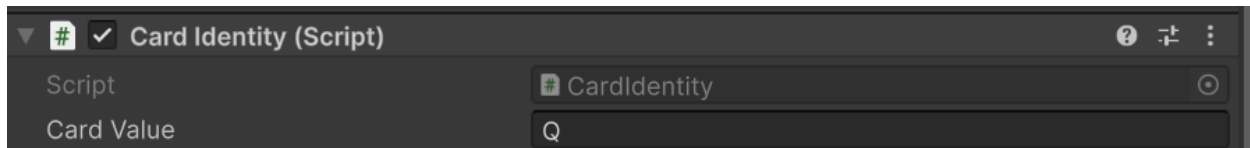


Figure 16: Card Identity Component

When the system successfully recognizes the card, it submits the card value and the card object together to "RoundGameManager.RequestPlayCard()", completing a play.

```
roundGameManager.RequestPlayCard(card.cardValue, card.gameObject);
```

After receiving a card-playing request, RoundGameManager first checks if the current round has ended. It will detect if the player is in the Photon room and if a local player exists.

Then, the system retrieves the card's PhotonView and checks if the card belongs to a local player. Only cards belonging to the player are allowed to be submitted. This check prevents players from submitting cards that don't belong to them.

```
int actor = PhotonNetwork.LocalPlayer.ActorNumber;
int cardViewId = cardPv.ViewID;
photonView.RPC(nameof(RPC_PlayCardOnMaster), RpcTarget.MasterClient, actor, cardValue, cardViewId);
if (info.Sender == null || info.Sender.ActorNumber != actor) return;
PhotonView cardPv = PhotonView.Find(cardViewId);
```

Card play requests are not processed locally but are instead sent through RPC to the MasterClient for unified processing. After receiving the request, the MasterClient verifies that the sender Actor matches the Actor in the request and finds the corresponding card object using PhotonView.Find.

```
if (!PhotonNetwork.IsMasterClient) return;

if (info.Sender == null || info.Sender.ActorNumber != actor) return;
PhotonView cardPv = PhotonView.Find(cardViewId);
```

To prevent the same card from being counted repeatedly due to multiple entries into the trigger zone, the system uses committedCardViewIds to record committed cards. If a card is submitted repeatedly, it is returned directly.

Each player initially has 5 cards in hand. MasterClient initializes each player's hand size as needed. If a player hasn't recorded their hand size yet, it's set to 5. When a player successfully plays a card, their hand size is reduced by 1.

```
if (!handCountByActor.ContainsKey(actor)) handCountByActor[actor] = 5;
handCountByActor[actor] = Mathf.Max(0, handCountByActor[actor] - 1);
photonView.RPC(nameof(RPC_PlayCommittedAll), RpcTarget.All, actor, cardValue, cardViewId);
```

If a player's hand size reaches 0, it means they've played all 5 cards, the round ends. And the system announces the player's victory.

```
if (handCountByActor[actor] == 0)
{
    roundEnded = true;
    photonView.RPC(nameof(RPC_RoundWinByEmptyHandAll), RpcTarget.All, actor);
}
```

After the winning result is broadcast to all clients, each client determines whether its local player is the winner and plays victory or defeat feedback accordingly. The sound effects are handled by PlayAudioFeedback.PlayWinLoseSfx(), which selects either a victory or defeat sound effect based on isWinner.

```
audioFeedback.PlayWinLoseSfx(isWinner);
```

Visual feedback is handled by WinEffectsManager.PlayEndGameEffects(), which plays fire or smoke animations based on the win or loss and displays the victory or defeat UI. Therefore, the complete closed loop of the normal turn-based card-playing mode is that player plays a card, system verification, deducts cards from hand, determines if all 5 cards have been played, and plays the win or loss result.

4.6.2 Mode 2: Player 1 Plays a Card, Player 2 Challenges



Figure 17: Mode 2

In the challenge mode, Player 1 plays a card first. After a successful play, the system broadcasts the result to all clients through “RPC_PlayCommittedAll()”, records the previous player number and the previous card. It indicated that there is a previous card that can be challenged. This step corresponds to the "Declare Queen, Challenge?" phase after Player 1 plays their card in the flowchart. And will play audio effect, “Card has been played”. This audio effect was created by us. We use app Voice changer to achieve it.^[25]

```
private void RPC_PlayCommittedAll(int actor, string cardValue, int cardViewId)
{
    LastPlayedActor = actor;
    LastPlayedCard = cardValue;
    HasPendingPlay = true;
    if (audioFeedback != null)
    {
        Debug.Log("Round Playing card submitted ");
        audioFeedback.PlayCardSubmittedSfx();
    }
}
```

If Player 2 chose to challenge, they would click the challenge button. The button script “ChallengeButtonTrigger” called “RoundGameManager.RequestChallenge()”, handing the challenge action over to the round manager.

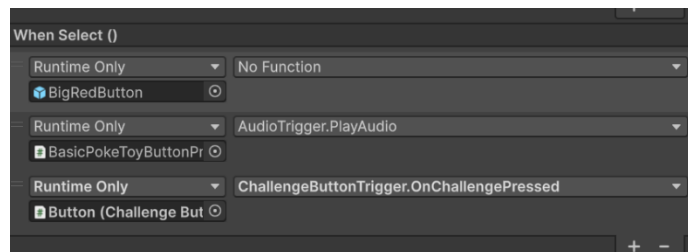


Figure 18: Challenge Button Conditions

The challenge request was also sent to the MasterClient for processing to avoid inconsistent results caused by different clients making their own judgments. The MasterClient reconfirmed its identity as the MasterClient, whether there was a card to be challenged, and whether the challenger's identity matched

the sender's. The system then checks if the previous card was a Queen based on the last played card recorded during the previous play. The face value of each card came from the card identity.

If the previous card was indeed a Q, then Player 1's challenge was unsuccessful, and Player 1 wins. If the previous card was not a Q, then Player 2's challenge was successful, and Player 2 wins. This winner selection was accomplished in code.

```
int winnerActor = lastCardIsQ ? playedActor : challengerActor;
```

The system then played a challenge result sound effect, followed by a win/loss sound effect and visual effects. After the challenge ends, the MasterClient called HandResetManager, causing all clients to reset their hand positions.

```
handResetManager.RequestResetAllFromMaster();
```

The HandResetManager then notifies all players through RPC to perform a local reset.

```
photonView.RPC(nameof(RPC_DoResetLocalOwnedHand), RpcTarget.All);
```

It moved the root hand objects of the local players back to their corresponding hand spawn points. `handRoot.SetPositionAndRotation(targetSpawn.position, targetSpawn.rotation);`

Therefore, the complete flow of the mode where Player 1 played a card and Player 2 challenges was, Player 1 plays a card, the system records the previous hand, Player 2 clicks to challenge, the MasterClient checks if it's a Q, determines the winner, plays feedback, and resets the hands.

4.6.3 Hurry Up Sound Effect Button

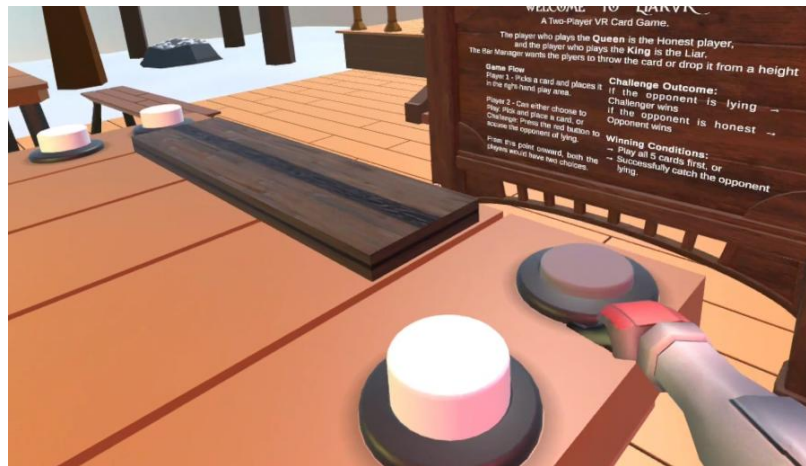


Figure 19: Hurry Up Sound Effect Button

To enhance the entertainment value, we added two sound effect buttons, "hurry up" and "lag? or just you?", allowing players to communicate with each other and adding to the fun. These sound effect buttons also trigger events through the network, broadcasting sound effects to all players.

4.7 Card Issue

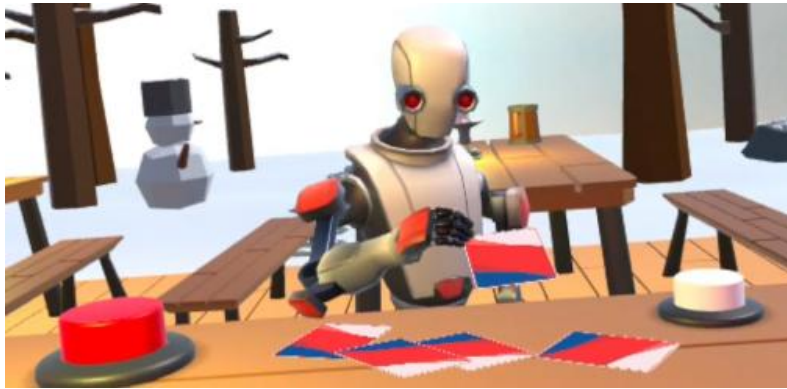


Figure 20: Card

After establishing the basic turn-based framework, we optimized the card-grabbing interaction to enhance the overall game experience. In early multiplayer testing, a serious visual synchronization issue arose. When a local player picked up a card, the card's tracking and grabbing behavior appeared perfectly normal from their perspective. However, from the perspective of a remote player, the card exhibited a violent and continuous up-and-down jumping in the opponent's hand.

We investigated the cause of this problem. Initially, we suspected a mesh rendering issue caused by the card using a 2D image prefab instead of a standard 3D cube. However, after replacing the card model with a standard 3D cube, the shaking phenomenon persisted. It lead us to conclude that the root cause was a conflict between the underlying grabbing interaction logic and network synchronization.

Then, we began searching for a breakthrough in the card's attached components. Using a controlled variable method of disabling components one by one, we debugged four potentially related parameters and component orientations. We tried Kinematic with selected, Owner Transfer Request, and Photon Transform View. Considering precedents regarding model synchronization, we speculated that other local physics components might be overriding Photon's network data. Therefore, we tried introducing the Photon Rigidbody View component to assist in synchronizing the rigid body state, but test results showed that this method was ineffective. During the troubleshooting process, we also referred to relevant technical discussions on Stack Overflow.^[26]

Finally, we focused on the gravity mechanism within the physics engine. In Unity VR interaction, when a player picks up an object, the interaction component attempts to override the object's displacement data. And when the object is released, the gravity component takes over again. With network synchronization, local interaction can completely suppress gravity. But on the remote, a conflict arises between data synchronization and the local physics engine. Specifically, the remote client's physics engine, based on gravity logic, determines that the object is not on a supporting surface and should fall downwards. However, the Photon Transform View component is constantly updating from the network and forcibly correcting the card's actual position when it is picked up at an extremely high frequency. The downward pull of gravity and the upward correction force of network synchronization frequently alternate on the remote end, visually manifesting as the card bouncing violently in the hand.

After clarifying the nature of the conflict, we provided a solution by writing control scripts. In the script, we dynamically controlled the properties of the card's rigidbody. The moment the card was grabbed by the player, the rigidbody was immediately canceled at the code level, thus temporarily disabling the card's gravity simulation during the grab. In this way, the physics engine on the remote end no longer calculated

its gravity fall, and the network position update data was successfully taken over. It completely resolved the conflicting physics logic between the two ends and restoring smoothness and stability to the network grab synchronization of the card.

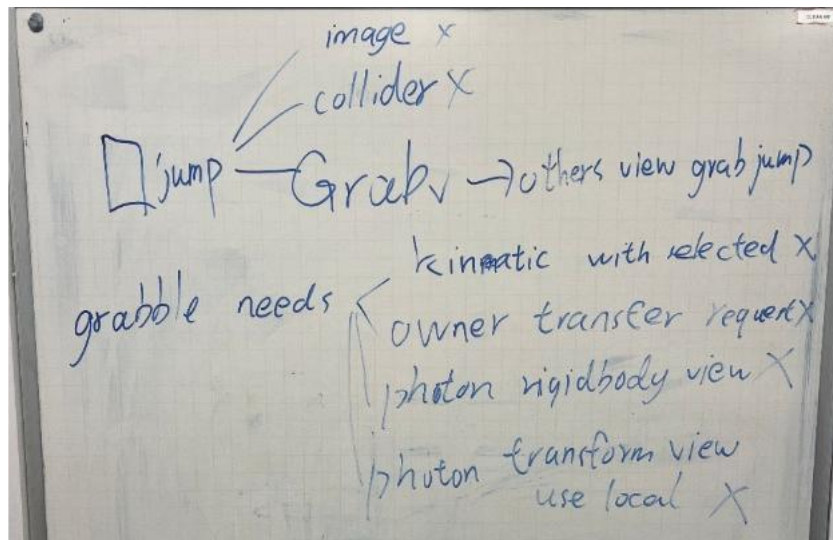


Figure 21: Card Grab Conflict Debugging

4.8 Test of Multiplayer Games by One Person

In the initial stages of the project, our testing environment consisted of a standard 3D Unity two-player game. Since the game required two players to enter the same scene simultaneously, we needed to open two Game Windows to simulate the perspectives and actions of each player. However, in actual development, frequently having two team members test simultaneously was cumbersome. And many basic functions only required observing whether two prefabs could be generated and synchronized correctly. Therefore, we began exploring how to test the two-player game in a single-player development environment.^[27]

We tried using the ParrelSync. This plugin allows you to open two Unity editor instances on the same computer, thus running two Unity Game Windows simultaneously. This allows us to simulate two clients on one computer, observing whether two player objects can enter the same room and their synchronization effects in the scene.

However, this method had some limitations. Because it required running two Unity editors simultaneously, our personal computers sometimes experience lag, and even occasional crashes. Therefore, while this method is suitable for rapid early testing of basic networking logic, it's not ideal for long-term, stable testing of full functionality.

After adding VR cameras, we tried another testing method: one client was built and run on the headset, while the other client was run directly in the Unity Editor by clicking "Play." Since we were using the Oculus RigOVR camera, we could enable the Oculus Simulator in the Editor to simulate a VR perspective. This allowed us to use only one headset to simultaneously observe and test the visuals and interactions of both players on the same computer.

This approach is helpful in the mid-stages of development because it reduces the reliance on two testers and two devices. It allowed for easier debugging of two-player synchronization, player generation.

However, in later stages, when some features fail to function correctly in the Unity Editor or require testing in a real VR headset environment. The single-computer, single-headset approach is no longer sufficient. At this point, we need to build and run the project separately on two VR headsets to conduct real two-player testing using both devices, confirming the final game's performance in a real VR environment.

4.9 End Game Sequence: Visual Effects

The purpose of choosing to create particle effects and work towards creation of visual effects was to keep the users more enthusiastic, to enhance player engagement, and for the user's emotional feedback at the end of game. As of our understanding, a proper outro to a game would make the players feel good about their experience, could push the players towards playing again, and our approach towards achieving that was by using visual effects created using the Unity's particle system. The aim was to create a satisfying end experience by creating visually appealing effects, engaging audio feedback with UI texts.

Based on our understanding, the implementation of visual effects could improve the overall quality of the game by placing a smooth transition between the competitive gameplay and the display of results. This would contribute towards a more polished game developed professionally.

4.9.1 Result: Victory

The victory sequence, which was developed using the Unity's particle system, would display a fire particle effect on the screen for the player who won the game. The fire particle effect was selected as it would visually represent victory and celebration combined with an engaging and celebratory audio to signal the victory, and a text saying "You Won. Yayyy!!!" would create a fully satisfying celebratory experience for the player who won the game.



Figure 22 : Victory Sequence

4.9.2 Result: Defeat

The defeat sequence, which was developed using the Unity's particle system, would display a smoke particle effect on the screen for the player who lost the game. The smoke particle effect was selected as it

would visually represent defeat with a motivational audio to signal the defeat, and a text saying “Your Victory Is Just Around The Corner” would create a motivational experience for the player.



Figure 23 : Defeat Sequence

4.9.3 Unity Particle System

Start Speed of the Particles: To control how fast the particles come into the scene.

Start Size of the Particles: To control how big the particles are.

Start Lifetime of the Particles: To control the lifetime of the particles, after which they would disappear.

Start Rotation of the System: To control the rotation of the whole system.

Emission: To control the number of particles generated over time.

Volumetric Shape: To control how the fire effect starts and how wide it expands to burn.

Colour Over Lifetime: To control the colour of the particles over their lifetime in the scene.

Size Over Lifetime: To control the size of the particles over their lifetime in the scene.

Noise: To add some randomness to the particle movement.

Texture Sheet Animation: To control the particle's texture and partial shape.

Maximum Particle Size: To control the maximum particle size. 1 – Full size, 0.5 - Half of default size.

4.9.4 Development of Fire Effect

The fire particle effect created using the Unity's native particle system, with an aim to simulate a real-world fire uses a co-related effect of three particle systems integrated as one system. The first particle system is to make the fire bigger, giving a real fire effect. The second particle system is to make the core body part of the fire look denser.

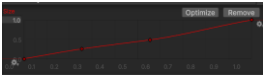
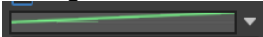
Module	Particle System 01	Particle System 02	Reason
Start Speed of the Particles	1.5 units	1 unit	To give the effect a sudden surge to simulate real-world fire
Start Size of Particles	1 meter (unit)	2.5 meters (units)	To keep the particle size small as it grows and big at its birth
Start Rotation of the System	Random between two constants: - 45 degrees to 45 degrees	Random between two constants: - 45 degrees to 45 degrees	To make the fire effect rotate rather than burning in a fixed spot in a straight upward way, to simulate a real-world fire
Emission	Rate over time: 200 particles per second	Rate over time: 200 particles per second	To create a dense roaring fire effect
Volumetric Shape	Cone Angle: 10 degrees Radius: 0.25 meters (units)	Cone Angle: 10 degrees Radius: 0.001 meters (units)	To create an effect of fire starting from one spot and expanding wider as it burns big.
Colour Over Lifetime			To simulate the colour of fire: Yellow → Orange → Black
Size Over Lifetime			To increase the size of the particles as the fire burns bigger, to simulate a real-world fire
Noise	Strength: 0.35 units Frequency: 0.5 units	Strength:  Frequency: 0.5 units	To introduce randomness into the particle's movement.
Texture Sheet Animation	3 x 3 Whole Sheet Grid	3 x 3 Whole Sheet Grid	To animate and randomize the texture of the particles in the fire effect.

Table 1: Fire Effect Development

4.9.5 Development of Smoke Effect

The smoke particle effect created using the Unity's native particle system, with an aim to simulate a real-world smoke uses a co-related effect of three particle systems integrated as one system. The first particle system is to make the smoke bigger, giving a real smoke effect. The second particle system is to make the core body part of the smoke look denser.

Module	Particle System 01	Particle System 02	Reason
Start Speed of the Particles	1.5 units	0.1 unit	To give a small and slow smoke effect to simulate real-world smoke

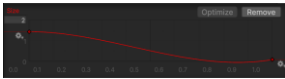
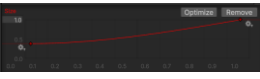
Start Lifetime of Particles	1.5 seconds	1 second	To make the particles disappear fast, as it would do in a real smoke
Start Rotation of the System	Random between two constants: - 45 degrees to 45 degrees	Random between two constants: - 45 degrees to 45 degrees	To make the smoke effect rotate rather than being in a fixed spot in a straight upward way, to simulate a real-world fire
Emission	Rate over time: 25 particles per second	Rate over time: 7 particles per second	To create small smoke effect
Volumetric Shape	Cone Angle: 15 degrees Radius: 0.0001 meters (units)	Cone Angle: 10 degrees Radius: 0.0001 meters (units)	To create an effect of smoke starting from one spot and expanding a bit as the smoke grows.
Colour Over Lifetime			To simulate the colour of smoke: White → Dark Grey
Size Over Lifetime			To increase the size of the particles as the smoke gets bigger, to simulate a real-world smoke
Texture Sheet Animation	2 x 2 Whole Sheet Grid	Not used	To animate and randomize the texture of the particles in the smoke effect.
Maximum Particle Size	0.5 units	0.5 units	To keep the smoke small

Table 2: Smoke Effect Development

5 | Critical Evaluation

Key finding: The real questionnaire data show a moderately positive VR presence experience (4.77/7), with strong visual-spatial presence but weak interface stability. This supports the need for Card-Box stabilization, clearer feedback, and reduced delay/discomfort in future iterations.

5.1 Data Source and Method

This evaluation uses the real Wenjuanxing questionnaire export file. Earlier cleaned spreadsheets and analysis notes were treated only as processing outputs, not as original data sources.

Additionally, participants also received the same 24-item presence questionnaire after using LiarVR, measuring immersion on a 1-7 scale (similar to how other Presence Questionnaires have been administered) about their virtual reality experience.^[19] Presence is appropriate for this prototype because VR quality depends on interactivity, sensory involvement, spatial perception, and the user's feeling of acting inside the environment.^{[28][29]}

Metric	Result
Valid responses	22
Q1-Q24 missing values	0
Straight-line response flags	0
Scale	1-7 scale
Reverse-scored items	Q14, Q17, Q18

Table 3 : the real questionnaire dataset and the basic data-quality checks used before analysis

5.2 Data Processing and Simple Model

Q14, Q17 and Q18 were used to evaluate three types of negative experience: delay (Q14), visual interference (Q17), and control-device interference (Q18). These three questions were reverse-scored by using the equation to correct for the negative impact of answering the questions incorrectly: corrected score = 8 - original score. After applying the correction, high scores tell consistently better experiences on all of the measures.

The overall score was calculated as the mean of corrected Q1-Q24. A presence index was then mapped to a 0-100 scale: Presence Index = ((mean score - 1) / 6) x 100.

Because the amount of records from our source was only 22, and there was no independent variable available for our predictions, an explanatory model was used instead of a predictive model.

Dimension	Items	Meaning
Control and Interaction	Q1, Q2, Q3, Q5, Q8, Q15, Q16, Q19, Q24	Control, natural interaction, task focus and object manipulation.
Visual and Spatial Presence	Q4, Q6, Q7, Q9, Q10, Q11, Q12, Q13	Visual involvement, spatial movement and object inspection.
Interface Stability	Q14R, Q17R, Q18R	Reverse-scored delay, visual interference and device interference.
Audio and Haptic Involvement	Q20, Q21, Q22, Q23	Sound involvement, localization and touch-based exploration.

Table 4 : how the 24 questionnaire items were grouped into four interpretable evaluation dimensions

5.3 Quantitative Results

Metric	Result
Overall Presence Mean	4.77 / 7
Overall Presence SD	0.92

Presence Index	62.91 / 100
Cronbach's alpha	0.916

Table 5 : The overall presence score and internal consistency of the questionnaire

Dimension	Mean / 7	SD	95% CI	Index / 100	Alpha
Control and Interaction	5.03	1.32	4.48-5.58	67.17	0.941
Visual and Spatial Presence	5.18	1.31	4.63-5.72	69.60	0.943
Interface Stability	2.91	1.51	2.28-3.54	31.82	0.950
Audio and Haptic Involvement	4.80	1.32	4.24-5.35	63.26	0.837

Table 6 : Comparison of the four dimension scores after reverse scoring

LiarVR Presence Questionnaire: Dimension Scores (N=22)

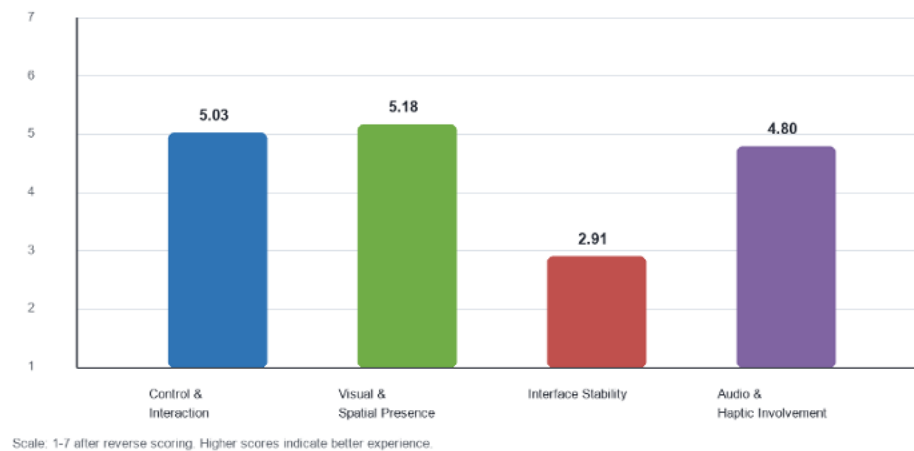


Figure 24 : Dimension scores calculated from the real Wenjuanxing questionnaire data

The strongest dimension was Visual and Spatial Presence (5.18/7), followed by Control and Interaction (5.03/7). This suggests that the tavern environment, object layout and hand-based card interaction supported a basic sense of VR presence. The weakest dimension was Interface Stability (2.91/7), indicating that delay, visual interference and control-device interference were the most important problems.

Lowest and Highest Questionnaire Items

Corrected mean score on 1-7 scale; lower items indicate main improvement areas.

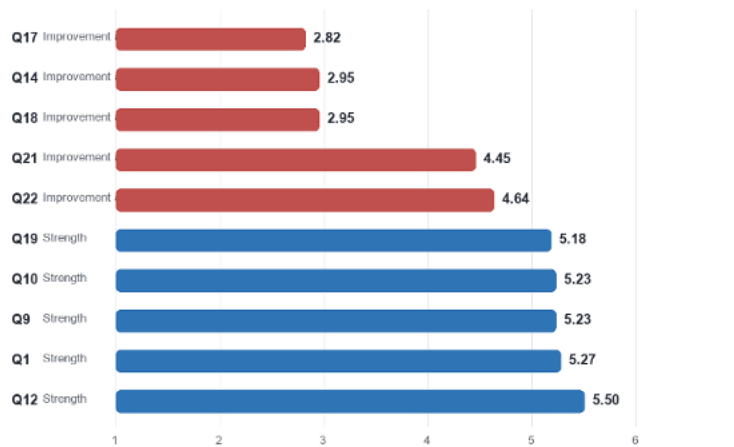


Figure 25 : Lowest and highest scoring items after reverse scoring

Item	Mean / 7	Interpretation
Q17	2.82	Reverse-scored problem item
Q14	2.95	Reverse-scored problem item
Q18	2.95	Reverse-scored problem item
Q21	4.45	Lower positive item
Q22	4.64	Lower positive item
Q12	5.50	Experience strength
Q1	5.27	Experience strength
Q9	5.23	Experience strength
Q10	5.23	Experience strength
Q19	5.18	Experience strength

Table 7 : the lowest and highest scoring questionnaire to show both strengths and improvement areas

5.4 Qualitative Feedback

There was little to write about for the open-ended comment types, which can be used to support an evaluation but should not be a primary means of evaluating. The main comments made about the evaluations were sound, tactile grabbing of cards, lag or skipping, dizziness and the recommendation for more multiple players to play together.

Theme	Evidence from responses	Interpretation
-------	-------------------------	----------------

Embodied interaction	Hand-based card grabbing	Hand-based card manipulation was noticed as an immersive interaction.
Performance issue	Lag or stuttering during operation	Lag or stuttering reduced interaction quality.
Comfort issue	Dizziness	Some players experienced VR discomfort.
Social interaction	Suggestion to add more multiplayer interaction	The multiplayer bluffing experience could be expanded.
Audio involvement	Sound was mentioned as immersive	Sound contributed to immersion for at least one participant.

Table 8 : summarizes the limited but useful open-ended feedback themes from participants

5.5 Critical Evaluation and Reflection

A questionnaire where asked about the results of the presence assessment show that whereas the presence for the project was moderate overall, the degree to which different aspects of the various components of the system contributed to this overall score was variable among the component parts. For example, the overall item with the highest strength, Visual and Spatial Presence, indicated that the tavern, table, items on it and how they were viewed from a spatial standpoint helped to support the overall sense of being in a virtual reality scene thus supporting the environmental and visual design of the space as having purpose beyond being a decorative environment and at the same time helping to frame the card game as an embodied social experience rather than as a flat digital card interface; and therefore create a sense of being in a real environment.^{[19][28][29]}

Several aspects of how to build a game are apparent from the players' responses about interaction and control. Players tend to realize that they can touch the cards and take some initial actions with them - this relates to how cards were set up for hand-based interaction, how cards are dealt/resetted and how players submit their cards to the card box. The challenge of the dealing animation representing an organized hand as opposed to an irregular or unstable physical condition of the cards helps create a clear visual understanding of the game state before the players start to engage with the cards. This is significantly important for VR where the way that objects are arranged, moved and respond to action can create an understanding of what users are able to do next with the objects.^{[17][30]}

The Introduction UI should also be a factor in evaluation, not just an element of the tutorial. The components of LiarVR are multiplayer connection; turn-taking; physical handling of the cards; playing cards and making challenges. For someone new to the game, this could be overwhelming to absorb at one time. A guidance panel is included to provide users with contextual information during actual gameplay regarding waiting for both players before playing a card, knowing whose turn it is, when to submit a card, and when challenges can occur. This is important because VR instructions should support action without forcing the player to memorize all rules before entering the scene.^[30]

The multiplayer aspect is also critical to the reflection. LiarVR is a 2-player bluff game therefore, it cannot be played independently of one another. Both players must exist with the same game state, knowing the order of play and able to react to one another's actions. The current user study measured presence and interaction quality only as a means to indirectly measure the gaming of bluffing with another player.

Research on social VR indicates social environments are affected by the quality of interaction, the ability of the user to embody the user within that environment and the time it takes for each user to respond.^[3] For multiplayer games to be evaluated satisfactorily, they must not only consider connection success, but also evaluate whether or not they create tension, trust, deception and must evaluate if they provide meaningful aware of opponents.

Of all the results received, the weakest - Interface Stability - is the strongest signal for needed improvements. This dimension scored much lower than other dimensions because it was mostly based on reverse-scored delay and visual interference and control device interference. Interface Stability does not mean the concept failed; however, it indicates that the primary barrier is that the VR interaction was not reliable and/or not comfortable for users. Users will often lose their sense of presence due to unstable object behavior or delayed feedback as they expect the virtual environment to respond consistently based on their physical movement.^{[31][32]} This finding is relevant to the Card-Box, but the reflection should not overemphasize the Card-Box as the only solution. Because the card has been submitted and has been secured in the box, our confidence level for determining when a card has been accepted and held is very high because it will be securely locked in place and cannot be submitted multiple times or be grabbed again (be accident) once accepted into the box. Thus, this relates to only one aspect of interface stability. The findings indicate that interface stability includes stability of the entire system, including: tracking of the controller, visual comfort, response timing, physics of the objects, and clarity of the feedback given to the user when an action happens.

Audio and haptic experiences were moderately immersive instead of strongly immersive. Some examples of open-ended feedback included sound and hand-based card grabbing as immersive. Sound identification and sound localization did not have the strongest ratings; therefore, audio may not have been a major factor in the player's sense of being present. Future work could consider using more obvious examples of spatial audio cues for card submission, challenge moments and turn changes to provide additional support for both immersion and usability, allowing players to understand what has occurred without relying only on the visual user interface.

Qualitative comments are limited but consistent with the quantitative trend so positive qualitative comments about the physicality of hand-based card grabbing support the idea that embodied interaction is an advantageous component of the design. Negative qualitative comments such as lag and dizziness aligned well with the low score of interface stability; because of the limited open-ended qualitative comments, they are best used to augment the quantitative data rather than serve as an exhaustive thematic analysis. Nevertheless, qualitative comments are useful because they align with the same priorities as the questionnaire: enhancing stability of the interactions, decreasing discomfort, and enhancing clarity of feedback.

All of the components of LiarVR constitute an integrated prototype as follows: A visual environment, a hand-based way to interact with cards (dealt as well), a use of a Card-Box, the use of an introduction guide, and the use of a multiplayer system. The biggest improvement that could be made to the component parts would be to create a greater sense of reliability and connectedness. The next update should continue building out LiarVR, while also improving the overall interaction loop by providing a clear introduction to each player, properly displaying cards for each player, providing opportunities for players to submit cards with confidence, indicating whether a player's submission was successfully submitted, providing synchronized results to each player's submission, and providing clear feedback as to why the players' challenge was declined.

5.6 Limitations and Future Work

While we've finally completed a basic version of a two-player VR card game, implementing multiplayer synchronization, VR body tracking, and turn-based logic. The project still has many limitations. Through this development, we've gained a clearer understanding of the current system's limitations and areas for future improvement.

First, the project's overall completion level is still leaning towards technical rather than a mature game. We are currently focusing on testing whether a two-player VR turn-based card game is feasible and whether VR body movements can be used for strategic gameplay. Although many functions are functional, stability and completeness are still insufficient. For example, multiplayer synchronization currently relies on Photon's basic synchronization mechanism, which works with a small number of players. But if we add more players, network stability is not high enough. Increased network latency leads to errors in card position synchronization and character movement synchronization. In actual gameplay, this latency directly affects players' judgment of actions and psychological game dynamics.

Secondly, character body tracking still faces many challenges. Although basic body movement synchronization is currently achievable, model scaling, pose initialization, and character calibration remain unstable. For example, character size may become inconsistent when switching between sitting and standing postures. This indicates that our understanding of the Meta Movement SDK and character retargeting is not yet deep enough. Future development aims to establish a more stable character calibration system, such as adding a unified height and pose initialization process before game start, rather than relying entirely on automatic scaling.

Furthermore, the current game interactions are still quite basic. While we've implemented card drawing, placement, and synchronization, the interaction methods still lack realism. For example, there's a lack of natural finger feedback when players draw cards. There was no more realistic physical representation between cards. And the play area only has simple collision detection. In VR games, the interactive experience directly impacts immersion, and our current interaction feels more like functional implementation than real operation. In the future, we could add more refined gesture recognition, card animations, and haptic feedback, allowing players to truly feel like they are "manipulating cards on a table," rather than just moving an object.

At the game rules level, the current system is relatively simple. We've primarily completed the two core mechanisms of taking turns and challenging, but the overall game flow is incomplete. For example, it lacks clearer UI, game status displays, and turn feedback. During gameplay, players sometimes don't know whose turn it is, whether they can challenge, or whether the game is over. This indicates that we are currently neglecting user experience design. If development continues, we hope to add a clearer UI system, such as turn prompts, card count displays, player status displays. It will make the entire game flow more complete. And user also have a better experience.

Furthermore, our current implementation of VR social experiences remains relatively limited. While players can see each other's movements, the connection between physical actions and game mechanics is not yet strong enough. Actions are always without truly influencing game strategy. So the psychological game between players is still not intense enough. In the future, we hope to further strengthen the role of action in the game, such as through facial tracking and heartbeat detection to enhance player observation and judgment. It can improve VR embodied interaction a true part of the gameplay, not just a visual representation.

References

- [1] WEI, X, JIN, X, Lin Kan, G, Yan, Y & Fan, M 2025, 'Systematic Literature Review of Using Virtual Reality as a Social Platform in HCI Community', Proceedings of the ACM on Human-Computer Interaction, vol. 9, no. 2, CSCW180, pp. 1-36. <https://doi.org/10.1145/3711078>.
- [2] Riches, S., Elghany, S., Garety, P., Rus-Calafell, M., & Valmaggia, L. (2019). Factors Affecting Sense of Presence in a Virtual Reality Social Environment: A Qualitative Study. *Cyberpsychology, Behavior, and Social Networking*, 22(4), 288-292.
- [3] MacArthur C, Kukshinov E, Harley D, Pawar T, Modi N and Nacke LE (2024) Experiential disparities in social VR: uncovering power dynamics and inequality. *Front. Virtual Real.* 5:1351794. doi: 10.3389/frvir.2024.1351794.
- [4] Twitch. (2025). Liar's Bar - Twitch. [online] Available at: <https://www.twitch.tv/directory/category/liars-bar> [Accessed 16 May 2026].
- [5] Mohan-Hickson, M. (2024). Sunderland Echo. [online] Sunderland Echo. Available at: <https://www.sunderlandecho.com/news/people/liars-bar-video-game-explained-4842922> [Accessed 16 May 2026].
- [6] to, C. (2026). Liar's Bar. [online] Video Games Wiki. Available at: https://wiki-of-games.fandom.com/wiki/Liar%27s_Bar [Accessed 16 May 2026].
- [7] OCULUS-SAMPLES. Unity-SpiritSling: Spirit Sling Tabletop [EB/OL]. GitHub. <https://github.com/oculus-samples/Unity-SpiritSling>
- [8] Unity Technologies. VR Multiplayer Template Quick Start Guide [EB/OL]. Unity Documentation, 2025. <https://docs.unity3d.com/Packages/com.unity.template.vr-multiplayerindex.html>
- [9] Unity. Build a Multiplayer VR Game in Unity 6 [EB/OL]. YouTube, 2025. <https://www.youtube.com/watch?v=i9GVZp9GZUE>.
- [10] immersive insiders. Setup VR Bodytracking with Meta Movement SDK [EB/OL]. YouTube. https://www.youtube.com/watch?v=ZCN_oi1vB0k.
- [11] bananadev2. Making MAPS and BETTER MOVEMENT! (Unity Multiplayer FPS Tutorial E02) [EB/OL]. YouTube. <https://www.youtube.com/watch?v=6G77nPCH2O4>.
- [12] Resolution Games. (2026). Demeo: Your New Favorite Tabletop Adventure Game. <https://www.resolutiongames.com/demeo>.
- [13] Innersloth. (2026). Among Us 3D. <https://www.innersloth.com/games/among-us-3d/>.
- [14] Divergent Realities. (2026). Cards & Tankards. <https://cardsandtankards.com/>.
- [15] Wei, X., Jin, X., Kan, G. L., Yan, Y., & Fan, M. (2025). Systematic Literature Review of Using Virtual Reality as a Social Platform in HCI Community. Proceedings of the ACM on Human-Computer Interaction, 9(CSCW), Article 180. <https://doi.org/10.1145/3711078>.
- [16] Biggs, B., Power, C., Wobbrock, J. O., & Hanson, V. L. (2025). Creating Non-Visual Non-Verbal Social Interactions in Virtual Reality. *Multimodal Technologies and Interaction*, 9(2), 25. <https://doi.org/10.3390/mti9020025>.

- [17] Bowman, D. A., Kruijff, E., LaViola, J. J., & Poupyrev, I. (2004). *_3D User Interfaces: Theory and Practice_*. <https://www.amazon.com/3D-User-Interfaces-Practice-Usability/dp/0134034325>
- [18] Jerald J .The VR Book: Human-Centered Design for Virtual Reality[J].Association for Computing Machinery and Morgan & Claypool, 2015.DOI:10.1145/2792790.2792807.
- [19] B. G. Witmer and M. J. Singer, "Measuring Presence in Virtual Environments: A Presence Questionnaire," in *Presence*, vol. 7, no. 3, pp. 225-240, June 1998, doi: 10.1162/105474698565686.
- [20] Blu. How To Make A Multiplayer Gorilla Tag Fan Game In 2026 [EB/OL]. YouTube. <https://www.youtube.com/watch?v=pWe18t5CwSE>.
- [21] GT. My New Gorilla Tag Rig [EB/OL]. Sketchfab. <https://sketchfab.com/3d-models/my-new-gorilla-tag-rig-ca9c13ab167a46ebaa03be6f568c0c25>.
- [22] Exit Games. Multiplayer Game Development Made Easy [EB/OL]. Photon Engine. <https://www.photonengine.com/>.
- [23] University of Bristol. Week 3 - lab tasks [EB/OL]. Blackboard Ultra. URL.
- [24] UT Unity Technologies. Robot Kyle | URP [EB/OL]. Unity Asset Store. <https://assetstore.unity.com/packages/3d/characters/robots/robot-kyle-urp-4696>.
- [25] Voice Changer - Online & Free [EB/OL]. CyberLink. <https://myedit.online/tw/audio-changer/voice-changer>.
- [26] VRDeveloper99. 2022. "Problem updating position of grabbable GameObject in VR Scene using PUN2 and Unity3D." Stack Overflow, October 15. <https://stackoverflow.com/questions/12345678>.
- [27] VeriorPies. ParrelSync: Test Multiplayer without Building [EB/OL]. GitHub. <https://github.com/VeriorPies/ParrelSync>.
- [28] Steuer, J. (1992). Defining virtual reality: Dimensions determining telepresence. *Journal of Communication*, 42(4), 73-93. <https://doi.org/10.1111/j.1460-2466.1992.tb00812.x>.
- [29] Slater, M. and Wilbur, S. (1997). A framework for immersive virtual environments: Speculations on the role of presence in virtual environments. *Presence: Teleoperators and Virtual Environments*, 6(6), 603-616. <https://doi.org/10.1162/pres.1997.6.6.603>.
- [30] Jerald, J. (2016). *The VR Book: Human-Centered Design for Virtual Reality*. ACM Books / Morgan & Claypool. <https://doi.org/10.1145/2792790>
- [31] Kennedy, R. S., Lane, N. E., Berbaum, K. S. and Lilienthal, M. G. (1993). Simulator Sickness Questionnaire: An enhanced method for quantifying simulator sickness. *The International Journal of Aviation Psychology*, 3(3), 203-220. https://doi.org/10.1207/s15327108ijap0303_3
- [32] LaViola, J. J. (2000). A discussion of cybersickness in virtual environments. *ACM SIGCHI Bulletin*, 32(1), 47-56.

Appendix A

Consent Form

**University of
BRISTOL**

PARTICIPANT CONSENT FORM

Please answer the following questions to the best of your knowledge

	YES	NO
HAVE YOU:		
• been given information explaining about the study?	☐	☐
• had an opportunity to ask questions and discuss this study?	☐	☐
• received satisfactory answers to all questions you asked?	☐	☐
• received enough information about the study for you to make a decision about your participation?	☐	☐
• ascertained that you don't have any known condition that prevents you from taking part in this study?	☐	☐
DO YOU UNDERSTAND:		
that you are free to withdraw from the study and free to withdraw your data prior to the conclusion of the experimental session		
• at any time?	☐	☐
• without having to give a reason for withdrawing?	☐	☐

I hereby fully and freely consent to my participation in this study

I understand the nature and purpose of the procedures involved in this study. These have been communicated to me on the information sheet accompanying this form.

I understand and acknowledge that the investigation is designed to promote scientific knowledge and that the University of Bristol will use the data I provide for no purpose other than teaching and research.

I understand the data I provide will be **anonymous**. No link will be made between my name or other identifying information and my study data.

I understand that the University of Bristol may use the data collected for this study in a future research project but that the conditions on this form under which I have provided the data will still apply.

I agree to the University of Bristol keeping and processing the data I have provided during the course of this study. I understand that this data will be used only for the purpose(s) set out in the information sheet, and my consent is conditional upon the University complying with its duties and obligations under GDPR.

Participant's signature: _____ Date: _____

Name in BLOCK Letters: _____

Figure 26: the Consent Form

Questionnaire

PRESENCE QUESTIONNAIRE
(Witmer & Singer, Vs. 3.0, Nov. 1994)*
Revised by the UQO Cyberpsychology Lab (2004)

Characterize your experience in the environment, by marking an "X" in the appropriate box of the 7-point scale, in accordance with the question content and descriptive labels. Please consider the entire scale when making your responses, as the intermediate levels may apply. Answer the questions independently in the order that they appear. Do not skip questions or return to a previous question to change your answer.

WITH REGARD TO THE EXPERIENCED ENVIRONMENT

1. How much were you able to control events?

NOT AT ALL | | | | | | | COMPLETELY

2. How responsive was the environment to actions that you initiated (or performed)?

NOT RESPONSIVE | | | | | | | COMPLETELY RESPONSIVE

3. How natural did your interactions with the environment seem?

EXTREMELY ARTIFICIAL | | | | | | | COMPLETELY NATURAL

4. How much did the visual aspects of the environment involve you?

NOT AT ALL | | | | | | | COMPLETELY

5. How natural was the mechanism which controlled movement through the environment?

EXTREMELY ARTIFICIAL | | | | | | | COMPLETELY NATURAL

6. How compelling was your sense of objects moving through space?

NOT AT ALL | | | | | | | VERY COMPELLING

7. How much did your experiences in the virtual environment seem consistent with your real world experiences?

NOT CONSISTENT | | | | | | | VERY CONSISTENT

8. Were you able to anticipate what would happen next in response to the actions that you performed?

NOT AT ALL | | | | | | | COMPLETELY

9. How completely were you able to actively survey or search the environment using vision?

NOT AT ALL | | | | | | | COMPLETELY

10. How compelling was your sense of moving around inside the virtual environment?

NOT COMPELLING | | | | | | | VERY COMPELLING

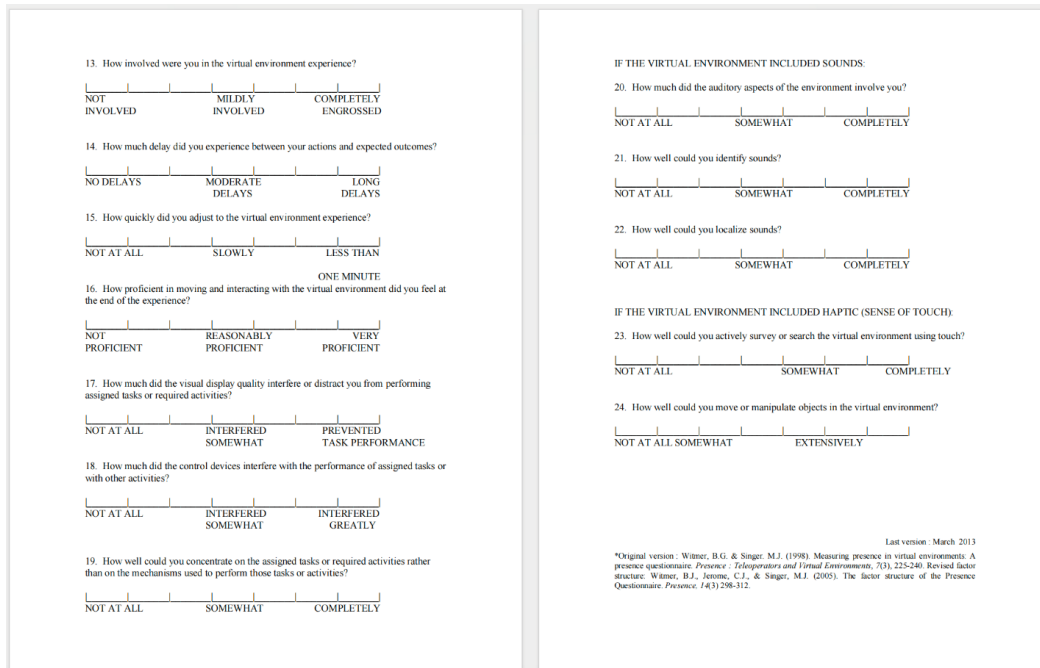
11. How closely were you able to examine objects?

NOT AT ALL | | | | | | | VERY CLOSELY

12. How well could you examine objects from multiple viewpoints?

NOT AT ALL | | | | | | | EXTENSIVELY

Figure 27: the Witmer and Singer Presence Questionnaire



13. How involved were you in the virtual environment experience?

NOT INVOLVED | MILDLY INVOLVED | COMPLETELY ENGROSSED

14. How much delay did you experience between your actions and expected outcomes?

NO DELAYS | MODERATE DELAYS | LONG DELAYS

15. How quickly did you adjust to the virtual environment experience?

NOT AT ALL | SLOWLY | LESS THAN ONE MINUTE

16. How proficient in moving and interacting with the virtual environment did you feel at the end of the experience?

NOT PROFICIENT | REASONABLY PROFICIENT | VERY PROFICIENT

17. How much did the visual display quality interfere or distract you from performing assigned tasks or required activities?

NOT AT ALL | INTERFERED SOMEWHAT | PREVENTED TASK PERFORMANCE

18. How much did the control devices interfere with the performance of assigned tasks or with other activities?

NOT AT ALL | INTERFERED SOMEWHAT | INTERFERED GREATLY

19. How well could you concentrate on the assigned tasks or required activities rather than on the mechanisms used to perform those tasks or activities?

NOT AT ALL | SOMEWHAT | COMPLETELY

IF THE VIRTUAL ENVIRONMENT INCLUDED SOUNDS:

20. How much did the auditory aspects of the environment involve you?

NOT AT ALL | SOMEWHAT | COMPLETELY

21. How well could you identify sounds?

NOT AT ALL | SOMEWHAT | COMPLETELY

22. How well could you localize sounds?

NOT AT ALL | SOMEWHAT | COMPLETELY

IF THE VIRTUAL ENVIRONMENT INCLUDED HAPTIC (SENSE OF TOUCH):

23. How well could you actively survey or search the virtual environment using touch?

NOT AT ALL | SOMEWHAT | COMPLETELY

24. How well could you move or manipulate objects in the virtual environment?

NOT AT ALL | SOMEWHAT | EXTENSIVELY

Last version: March 2013
 *Original version: Witmer, B.G. & Singer, M.J. (1998). Measuring presence in virtual environments: A presence questionnaire. *Presence - Teleoperators and Virtual Environments*, 7(3), 225-240. Revised factor structure: Witmer, B.J., Jerome, C.J., & Singer, M.J. (2005). The factor structure of the Presence Questionnaire. *Presence*, 14(3) 298-312.

Figure 28: the Witmer and Singer Presence Questionnaire

Interview Questions

1. What was your overall impression of LiarVR after playing it?
2. Which part of the experience felt most engaging or memorable, and why?
3. Which part of the experience felt most confusing or difficult?
4. Did you understand the basic goal and rules of the game before your first turn?
5. Did the introduction panel explain the turn-taking and challenge rule clearly enough?
6. Was the five-card hand layout easy to see, understand and use?
7. Did the card dealing/reset process make the start of the round feel clear?
8. Was it easy to pick up and place a card in VR?
9. Was it clear where you needed to place the card during your turn?
10. Did the play box give enough feedback when a card was accepted?
11. Did you understand when and why you could challenge the other player?
12. Did the game create tension or uncertainty when deciding whether to challenge?
13. Did you feel present at the virtual table with the other player?
14. Did you experience any discomfort, dizziness, eye strain or fatigue during the session?
15. What is the most important improvement you would suggest for the next version?